

平均流策略优化

Xiaoyi Dong^{1 2} Xi Sheryl Zhang¹ Jian Cheng^{1 3 4}

摘要

扩散模型近期已成为在线强化学习 (RL) 中富有表现力的策略表示。然而，其迭代生成过程带来了显著的训练与推理开销。为克服这一局限，本文提出使用平均流模型 (MeanFlow) ——一类少步基于流的生成模型——来表示策略，以提升相较于基于扩散的强化学习方法的训练与推理效率。为促进探索，本文在最大熵强化学习框架下通过软策略迭代优化平均流策略，并解决平均流策略特有的两个关键挑战：动作似然评估与软策略改进。在 MuJoCo、DeepMind 控制套件和 HumanoidBench 基准上的实验表明，本文方法——平均流策略优化 (MFPO) ——在性能上达到或超过当前基于扩散的基线，同时显著减少了训练与推理时间。代码见 <https://github.com/dongxiaoyi-xyz/MFPO>。

表示最优策略——其中一个必须是确定性的——其有限的表达能力往往导致低效的探索，从而阻碍策略优化。具体而言，在复杂控制任务中，奖励景观通常是多模态的。然而，高斯策略和添加高斯噪声的确定性策略本质上是单模态的，将探索限制在每个状态的单一动作区域。因此，其他潜在的高奖励区域可能被忽视，导致智能体陷入次优的局部最优解。

近期研究引入了扩散模型和流模型作为策略表示，以解决这一局限性 (Yang 等, 2023; Wang 等, 2024; Ding 等, 2024)。通过迭代过程逐步将噪声转化为动作，这些生成式策略能够建模高度复杂和多模态的动作分布，从而改善探索并在在线强化学习中取得显著的实证性能。然而，其实际应用受到多步迭代生成的高计算成本制约，与传统单步策略相比，这大幅增加了训练和推理时间。

为缓解这一效率瓶颈，本文采用近期提出的平均流模型 (MeanFlow Models) (Geng 等, 2025) 作为策略表示。通过将学习目标从瞬时速度场转变为沿采样轨迹的平均速度，平均流模型在粗时间离散化下显著降低了离散化误差，从而仅需少量采样步骤即可实现高质量生成。因此，少步平均流策略保留了建模复杂多模态动作分布的能力，有助于在多个有前景的动作区域进行高效探索，同时显著降低基于扩散的强化学习方法中的计算开销。

此外，遵循最大熵 (MaxEnt) 强化学习范式，本文将策略熵纳入优化目标，以鼓励对状态-动作空间进行充分探索，并采用软策略迭代来优化平均流策略。然而，与其他基于扩散的方法类似 (Celik 等, 2025; Dong 等, 2025; Ma 等, 2025)，将软策略迭代应用于平均流策略面临两大挑战。首先，评估软 Q 函数需要计算平均流策略的动作似然，其精确表达式

1. 引言

在线强化学习 (Online Reinforcement Learning, RL)，即智能体通过与环境的交互来学习决策策略，已被证明是解决连续控制任务的一种强大框架 (Lillicrap 等, 2015; Schulman 等, 2015; 2017; Haarnoja 等, 2018; Fujimoto 等, 2018)。现有的大多数方法将策略参数化为从状态到确定性动作或动作空间上高斯分布的映射。尽管这类策略类别在理论上足以 $^1C^2$ DL，中国科学院自动化研究所 ² 中国科学院大学人工智能学院 ³ AiRiA ⁴ 中国科学院大学未来技术学院。通讯作者：程健 < jcheng@nlpr.ia.ac.cn >。

Proceedings of the 43rd International Conference on Machine Learning, Seoul, South Korea. PMLR 306, 2026. Copyright 2026 by the author(s).

涉及对瞬时速度场散度的不可解积分。其次，标准平均流训练依赖于目标分布的样本，而在强化学习环境中通常无法获得这些样本。

为应对这些挑战，本文提出了一种平均散度网络来近似动作似然中难以处理的积分，并引入了一种自适应瞬时速度估计方法，为平均流策略构建可处理的训练目标。由此产生的算法被称为平均流策略优化（Mean Flow Policy Optimization, MFPO），能够高效且原则性地将平均流模型集成到最大熵强化学习框架中。我们在 MuJoCo、DeepMind 控制套件和 HumanoidBench 的基准任务上评估了 MFPO。实验结果表明，MFPO 达到或超越了现有基于扩散的强化学习算法的性能，同时所需的采样步骤显著减少，训练和推理时间也大幅缩短。

2. 相关工作

扩散与流模型。扩散模型（Diffusion Models）（Sohl-Dickstein 等，2015）凭借其训练稳定性和强大的经验性能，已成为生成建模中的主导范式（Dhariwal 和 Nichol, 2021; Saharia 等，2022; Brooks 等，2024）。这些模型通过添加高斯噪声，逐步将数据分布转化为可处理的先验分布，然后训练神经网络逆转该过程以生成样本。逆向过程可表述为一系列高斯转移、一个随机微分方程（SDE）或一个等价的常微分方程（ODE）（Ho 等，2020; Song 和 Ermon, 2019; Song 等，2021）。流匹配（Flow Matching）方法通过直接建模相应常微分方程的速度场，进一步扩展了这一框架（Liu 等，2022; Lipman 等，2023）。由于基于扩散和基于流的方法具有相似的公式表述和实用实现，本文将它们统称为扩散模型。

尽管扩散模型具备强大的生成能力，但其迭代采样过程导致推理时间较长。因此，一系列研究致力于在不牺牲性能的前提下减少采样步数。基于蒸馏的方法（Salimans & Ho, 2022; Sauer et al., 2024）训练一个少步学生模型来模仿预训练的多步教师模型的输出。DPM-Solver（Lu et al., 2022; 2025）通过利用扩散常微分方程的结构设计了专门的高阶常微分方程求解器。一致性模型（Consistency Models）（Song et al., 2023; Song & Dhariwal, 2024）学习将常微分方程轨迹上的点直接映射到其起点，并在训练过程中强制不同时间点的预测保持一致。捷径模型（Shortcut Models）（Frans et al., 2025）将步长作为额外条件，并施加自洽性目

标，以确保使用较大步长的模型与使用较小步长的模型行为一致。

最近，与标准流模型中学习瞬时速度场不同，平均流模型（MeanFlow Models）（Geng et al., 2025）提出对平均速度场进行建模，该平均速度场定义为瞬时速度的时间积分除以相应的时间间隔。当平均速度场被准确学习后，即使使用非常少的采样步数，也能生成无离散化误差的样本。与竞争方法相比，平均流模型实现了更优的样本质量，并且可以从头开始训练，无需预训练、蒸馏或课程学习。因此，本文采用平均流模型作为策略表示，以在表达能力、推理速度和简洁性之间取得有效平衡。

在线强化学习的扩散策略。在在线强化学习（RL）中采用扩散策略的趋势日益增长。除扩散策略优化外，该场景下的一个关键挑战是如何有效平衡探索与利用。DIPO（Yang 等，2023）克隆回放缓冲区中存储的动作，并通过施加 Q 函数梯度来改进这些动作。QVPO（Ding 等，2024）和 FPMD（Chen 等，2025）利用 Q 函数对生成动作上的扩散/流损失进行加权以实现策略改进，QVPO 还引入了一个针对均匀采样动作的额外扩散损失作为正则化项以鼓励探索。GenPO（Ding 等，2025）构建了一个可逆扩散模型，其对数似然可通过变量替换公式精确计算，从而能够与 PPO 等经典在线策略 RL 算法兼容。FlowRL（Lv 等，2025）将 Q 损失与由最优 Q 函数加权的扩散损失相结合，旨在最优行为策略的邻域内优化策略。QSM（Psenka 等，2024）使用 Q 函数的梯度作为得分函数的回归目标。DACER（Wang 等，2024）通过整个扩散链反向传播 Q 函数梯度以优化策略参数。QSM 和 DACER 都依赖于向生成动作中注入额外高斯噪声来促进探索。

与通过添加高斯噪声在当前策略附近进行启发式探索不同，最大熵强化学习为平衡探索与利用提供了一个有原则的框架。然而，在最大熵目标下优化扩散策略并非易事，因为对于基于扩散的策略，熵计算和软策略改进都是不可解的。DIME（Celik 等，2025）推导出策略熵的一个下界，并通过扩散链反向传播梯度来更新策略。为了近似由 Q 函数诱导的玻尔兹曼策略，MaxEntDP 和 SDAC（Dong 等，2025; Ma 等，2025）从高斯分布中采样干净动作

基于噪声动作的条件分布，并利用 Q 函数对扩散损失进行加权。RFM (Li 等人, 2026 年) 使用一类后验均值估计器构建策略训练目标，并引入基于朗之万-斯坦算子的控制变量以降低估计量方差。此外，MaxEntDP 通过计算代价高昂的数值积分来估计策略熵。然而，当仅使用少量采样步骤时，这些方法性能会下降。具体而言，DIME 采用的熵下界在小步数预算下变得松弛，这可能对策略学习产生不利影响。MaxEntDP、SDAC 和 RFM 在仅用有限数量的采样步骤求解扩散常微分方程时，会遭受显著的离散化误差。最近，SAC-Flow (Zhang 等人, 2026 年) 使用序列架构（如门控循环单元或变换器）参数化扩散策略，并在软演员-评论家框架 (Haarnoja 等人, 2018 年) 内对其进行优化。尽管这种设计提高了训练稳定性，但它依赖于特定的架构选择，从而增加了实现复杂度。

目前，大多数基于扩散的在线强化学习方法需要 10–20 个采样步骤才能取得较强的经验性能，导致训练和推理速度明显慢于基于传统单步策略的方法。本文研究使用少步 MeanFlow 策略来提高训练和推理效率，并在最大熵框架下对其进行优化，以实现高效探索。

3. 预备知识

3.1. 最大熵强化学习

本文考虑连续动作空间中的策略学习。环境被建模为马尔可夫决策过程 (Markov Decision Process, MDP)，由元组 $(\mathcal{S}, \mathcal{A}, p, r, \rho_0, \gamma)$ 定义，其中 $\mathcal{S}$ 和 $\mathcal{A}$ 分别表示状态空间和动作空间。转移动力学 $p: \mathcal{S} \times \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}^+$ 定义了给定当前状态 $\mathbf{s}_t \in \mathcal{S}$ 和动作 $\mathbf{a}_t \in \mathcal{A}$ ， $r: \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$ 的条件下，下一状态 $\mathbf{s}_{t+1} \in \mathcal{S}$ 的概率密度函数。 $[r_{\min}, r_{\max}]$ 是有界奖励函数， ρ_0 表示初始状态分布， $\gamma \in [0, 1]$ 是折扣因子。给定策略 $\pi(\mathbf{a}_t|\mathbf{s}_t)$ ，本文将 π 诱导的边际状态-动作分布记为 $\rho_\pi(\mathbf{s}_t, \mathbf{a}_t)$ 。为记号方便，时间 t 处的奖励 $r(\mathbf{s}_t, \mathbf{a}_t)$ 简写为 r_t 。

最大熵 (MaxEnt) 强化学习通过引入策略熵来增强标准强化学习目标，以鼓励学习随机策略。所得目标为

$$J(\pi) = \sum_{t=0}^{\infty} \gamma^t \mathbb{E}_{\rho_\pi} [r_t + \alpha \mathcal{H}(\pi(\cdot|\mathbf{s}_t))], \quad (1)$$

其中 $\mathcal{H}(\pi(\cdot|\mathbf{s}_t)) = \mathbb{E}_{\mathbf{a}_t \sim \pi(\cdot|\mathbf{s}_t)} [-\log \pi(\mathbf{a}_t|\mathbf{s}_t)]$ 是策略

熵与 $\alpha \in \mathbb{R}^+$ 是控制奖励最大化与熵正则化之间权衡的温度参数。在该目标下，我们关注策略 π 的软 Q 函数，其定义为

$$Q^\pi(\mathbf{s}_t, \mathbf{a}_t) = r_t + \sum_{l=1}^{\infty} \gamma^l \mathbb{E}_{\rho_\pi} [r_{t+l} + \alpha \mathcal{H}(\pi(\cdot|\mathbf{s}_{t+l}))]. \quad (2)$$

3.2. 软策略迭代

最大熵强化学习目标可通过应用软策略迭代 (Haarnoja 等, 2018) 来实现。该算法在策略评估与策略改进之间交替进行，并在一定假设下可收敛至最优最大熵策略。在策略评估步骤中，当前策略 π 的软 Q 函数通过反复应用软贝尔曼算子直至收敛来学习：

$$\mathcal{T}^\pi Q(\mathbf{s}_t, \mathbf{a}_t) \triangleq r_t + \gamma \mathbb{E}[Q(\mathbf{s}_{t+1}, \mathbf{a}_{t+1}) - \alpha \log \pi(\mathbf{a}_{t+1}|\mathbf{s}_{t+1})]. \quad (3)$$

在策略改进步骤中，通过将学习到的 Q 函数所诱导的玻尔兹曼分布投影到策略集 Π 上，并最小化库尔贝克-莱布勒散度，从而获得新策略：

$$\pi_{\text{new}} = \arg \min_{\pi \in \Pi} \text{D}_{\text{KL}} \left(\pi(\cdot|\mathbf{s}) \left\| \frac{\exp(\frac{1}{\alpha} Q^{\pi_{\text{old}}}(\mathbf{s}, \cdot))}{Z(\mathbf{s})} \right) \right), \quad (4)$$

其中 $Z(\mathbf{s})$ 表示归一化玻尔兹曼分布的配分函数。可以证明，对于所有 $(\mathbf{s}, \mathbf{a}) \in \mathcal{S} \times \mathcal{A}$ ，

$Q^{\pi_{\text{new}}}(\mathbf{s}, \mathbf{a}) \geq Q^{\pi_{\text{old}}}(\mathbf{s}, \mathbf{a})$ 成立，表明新策略优于旧策略。通过在策略评估与策略改进之间反复交替，算法将收敛至策略集 Π 中的最优最大熵策略 (Haarnoja 等, 2018)。

3.3. 流匹配与平均流模型

流匹配是一类学习两个概率分布之间连续流的生成模型。形式上，令 $p_1(\mathbf{x}_1)$ 表示简单先验分布（例如标准高斯分布）， $p_0(\mathbf{x}_0)$ 表示数据分布。流匹配旨在学习一个时间相关的速度场 $\mathbf{v}(\mathbf{x}, t): \mathbb{R}^d \times [0, 1] \rightarrow \mathbb{R}^d$ ，该速度场定义了常微分方程

$$\frac{d\mathbf{x}_t}{dt} = \mathbf{v}(\mathbf{x}_t, t).$$

所诱导的流应将 $t=1$ 时刻的先验分布 p_1 传输至 $t=0$ 时刻的数据分布 p_0 。可行的流路径可构造为 $\mathbf{x}_t = a_t \mathbf{x}_0 + b_t \mathbf{x}_1$ ，其中 a_t 和 b_t 是预定义的插值调度。本文采用常用的线性调度 $a_t = 1-t$ 和 $b_t = t$ 。相应地，条件速度场由 $\mathbf{v}_t(\mathbf{x}_t|\mathbf{x}_0) = \frac{d\mathbf{x}_t}{dt}$ 给出。随后训练参数化神经网络 $\mathbf{v}_\theta$ ，通过 $\mathbf{x}_1 - \mathbf{x}_0$ 对条件速度进行边缘化来近似边缘速度场。由此得到的条件流匹配 (CFM) 损失为

$$\mathcal{L}_{\text{CFM}}(\theta) = \mathbb{E}_{t, \mathbf{x}_0, \mathbf{x}_1} \|\mathbf{v}_\theta(\mathbf{x}_t, t) - \mathbf{v}_t(\mathbf{x}_t|\mathbf{x}_0)\|^2. \quad (5)$$

训练完成后，生成新样本的过程为：首先采样 $\mathbf{x}_1 \sim p_1$ ，然后从 $t = 1$ 到 $t = 0$ 沿时间反向求解学习到的常微分方程。其解可写为 $\mathbf{x}_0 = \mathbf{x}_1 - \int_0^1 \mathbf{v}(\mathbf{x}_t, t) dt$ 。在实际中，一阶欧拉求解器使用 T 个离散步骤来近似该积分：

$$\mathbf{x}_{t_{i-1}} = \mathbf{x}_{t_i} - (t_i - t_{i-1})\mathbf{v}(\mathbf{x}_{t_i}, t_i),$$

其中 $\{t_i\}_{i=0}^T$ 是 $[0, 1]$ 中的一个递增序列。尽管在步数足够多时有效，但当 T 较小时，欧拉方法会产生较大的离散化误差。

为解决这一局限性，平均流模型 (MeanFlow Models) (Geng 等人, 2025) 提出直接建模平均速度场，其定义为时间积分瞬时速度与时间间隔之比：

$$\mathbf{u}(\mathbf{x}_t, r, t) \triangleq \frac{1}{t-r} \int_r^t \mathbf{v}(\mathbf{x}_\tau, \tau) d\tau. \quad (6)$$

基于此定义，瞬时速度场与平均速度场满足平均流恒等式：

$$\mathbf{u}(\mathbf{x}_t, r, t) = \mathbf{v}(\mathbf{x}_t, t) - (t-r) \frac{d}{dt} \mathbf{u}(\mathbf{x}_t, r, t), \quad (7)$$

其中时间导数由 $d \frac{d}{dt} \mathbf{u}(\mathbf{x}_t, r, t) = \mathbf{v}(\mathbf{x}_t, t) \partial_{\mathbf{x}} \mathbf{u} + \partial_t \mathbf{u}$ 给出。式 (7) 的右侧被用作平均速度网络 $\mathbf{u}_\theta$ 的训练目标。训练目标为

$$\mathcal{L}(\theta) = \mathbb{E}_{r, t, \mathbf{x}_t} \|\mathbf{u}_\theta(\mathbf{x}_t, r, t) - \text{sg}(\mathbf{u}_{\text{tgt}})\|_2^2, \quad (8)$$

$$\mathbf{u}_{\text{tgt}} = \mathbf{v}(\mathbf{x}_t, t) - (t-r)(\mathbf{v}(\mathbf{x}_t, t) \partial_{\mathbf{x}} \mathbf{u}_\theta + \partial_t \mathbf{u}_\theta),$$

其中 $\text{sg}(\cdot)$ 表示停止梯度算子。在实际中，边缘速度场可以用其无偏估计量——条件速度场——来替代，以计算训练目标。训练完成后，采样可以通过用平均速度替换时间积分来实现：

$$\mathbf{x}_r = \mathbf{x}_t - (t-r)\mathbf{u}(\mathbf{x}_t, r, t).$$

与瞬时速度场不同，对平均速度进行积分能够准确近似轨迹积分，而与采样步数无关。

4. 方法

本文采用平均流模型作为策略表示，并利用软策略迭代优化最大熵强化学习目标。首先在第 4.1 节给出平均流策略的公式化描述。为解决将其融入软策略迭代所带来的挑战，在第 4.2 节引入平均散度网络以近似平均流策略的动作似然，并在第 4.3 节提出自适应瞬时速度估计方法以实现有效的策略改进。最后，在第 4.4 节描述所提算法的实用实现。

4.1. 平均流策略表示

形式上，我们将策略表示为常微分方程的解

$$\frac{d\mathbf{a}_t}{dt} = \mathbf{v}(\mathbf{s}, \mathbf{a}_t, t). \quad (9)$$

从先验样本 $\mathbf{a}_1 \sim p_1$ 出发，解可由

$\mathbf{a}_0 = \mathbf{a}_1 - \int_0^1 \mathbf{v}(\mathbf{s}, \mathbf{a}_t, t) dt$ 给出。注意本文中变量 t 表示常微分方程时间而非强化学习时间索引，除非另有说明。

$\mathbf{v}$ 被称为瞬时速度场，相应的平均速度场定义为

$$\mathbf{u}(\mathbf{s}, \mathbf{a}_t, r, t) = \frac{1}{t-r} \int_r^t \mathbf{v}(\mathbf{s}, \mathbf{a}_\tau, \tau) d\tau. \quad (10)$$

一个由 θ 参数化的神经网络 $\mathbf{u}_\theta$ 被训练以拟合平均速度场。根据式 (10)，通过设置

$$r = t: \mathbf{v}_\theta(\mathbf{s}, \mathbf{a}_t, t) = \mathbf{u}_\theta(\mathbf{s}, \mathbf{a}_t, t, t)$$

可以得到参数化的瞬时速度场。训练完成后，通过在有限数量的采样步骤上累积平均速度来生成动作，这作为瞬时速度积分的近似：

$$\mathbf{a}_{t_{i-1}} = \mathbf{a}_{t_i} - \frac{1}{T} \mathbf{u}_\theta(\mathbf{s}, \mathbf{a}_{t_i}, t_{i-1}, t_i), \quad i = T, \dots, 1, \quad (11)$$

其中 T 为采样步数，且 $t_i = \frac{i}{T}$ 。我们将以状态 $\mathbf{s}$ 为条件的诱导动作分布称为均值流策略，并记为 $\pi_\theta(\mathbf{a}_0 | \mathbf{s})$ 。

4.2. 训练动作平均散度网络似然近似

利用瞬时变量替换公式 (Chen 等人, 2018)，MeanFlow 策略的动作似然可以表示为

$$\log \pi_\theta(\mathbf{a}_0 | \mathbf{s}) = \log p_1(\mathbf{a}_1) + \int_0^1 \nabla \cdot \mathbf{v}_\theta(\mathbf{s}, \mathbf{a}_t, t) dt. \quad (12)$$

这里， $\nabla \cdot$ 表示关于动作变量的散度算子，可以写成瞬时速度场的雅可比矩阵的迹：

$$\nabla \cdot \mathbf{v}_\theta(\mathbf{s}, \mathbf{a}_t, t) = \text{tr} \left(\frac{\partial \mathbf{v}_\theta(\mathbf{s}, \mathbf{a}_t, t)}{\partial \mathbf{a}_t} \right). \quad (13)$$

计算动作似然的一种朴素方法是显式地计算式 (13) 中的雅可比矩阵，并对散度进行数值时间积分。然而，雅可比矩阵计算和时间积分在计算上都很昂贵，这会显著减慢软策略迭代。因此，我们寻求一种更高效的似然估计方法。

受 MeanFlow 模型中平均速度概念的启发，我们引入一个平均散度网络 δ_ω 来

近似时间平均散度：

$$\delta(\mathbf{s}, \mathbf{a}_t, r, t) = \frac{1}{t-r} \int_r^t \nabla \cdot \mathbf{v}_\theta(\mathbf{s}, \mathbf{a}_\tau, \tau) d\tau. \quad (14)$$

这个定义与平均速度场的定义类似，只是将瞬时速度替换为瞬时散度。根据 MeanFlow 理论，训练平均散度网络需要瞬时散度的无偏估计量。

直接计算雅可比矩阵的迹需要 d 次网络反向传播，其中 d 是动作维度。相反，我们采用 Skilling - Hutchinson 迹估计器 (Skilling, 1989; Hutchinson, 1989)：

$$\epsilon^\top \frac{\partial \mathbf{v}_\theta(\mathbf{s}, \mathbf{a}_t, t)}{\partial \mathbf{a}_t} \epsilon, \quad (15)$$

其中 ϵ 从满足 $\mathbb{E}[\epsilon] = \mathbf{0}$ 和 $\text{Cov}(\epsilon) = I$ 的分布中采样。该估计器是无偏的，并且可以使用前向模式自动微分通过单次前向传播高效计算。在实践中，对多个样本取平均可以降低方差并提高训练稳定性。得到的估计器为

$$\widehat{\text{div}}(\mathbf{s}, \mathbf{a}_t, t, \epsilon_{1:N}) = \frac{1}{N} \sum_{i=1}^N \epsilon_i^\top \frac{\partial \mathbf{v}_\theta(\mathbf{s}, \mathbf{a}_t, t)}{\partial \mathbf{a}_t} \epsilon_i, \quad (16)$$

$\{\epsilon_i\}$ 其中 $i=1 \sim p(\epsilon)$ ，且 N 表示用于发散估计的样本数量。

在构建平均流损失之后，平均发散网络的训练目标为

$$\mathcal{L}(\omega) = \mathbb{E}_{\mathbf{s}, r, t, \mathbf{a}_t, \epsilon_{1:N}} \|\delta_\omega(\mathbf{s}, \mathbf{a}_t, r, t) - \text{sg}(\delta_{\text{tgt}})\|_2^2, \quad (17)$$

其中

$$\delta_{\text{tgt}} = \widehat{\text{div}} - (t-r) (\mathbf{u}_\theta(\mathbf{s}, \mathbf{a}_t, t, t) \partial_{\mathbf{a}} \delta_\omega + \partial_t \delta_\omega).$$

完整推导见附录 B.1。训练完成后，平均发散网络可纳入平均流采样过程，以高效计算动作似然：

$$\log \pi_\theta(\mathbf{a}_0 | \mathbf{s}) = \log p_1(\mathbf{a}_1) + \frac{1}{T} \sum_{i=1}^T \delta_\omega(\mathbf{s}, \mathbf{a}_{t_i}, t_{i-1}, t_i), \quad (18)$$

其中 $\{\mathbf{a}_{t_i}\}_{i=0}^T$ 是使用公式 11 生成的样本。平均发散网络和似然感知采样过程的训练步骤分别总结在算法 1 和算法 2 中。经验表明，本文提出的平均发散网络在仅增加 5% 训练时间的情况下，提供了准确的似然估计。

4.3. 自适应瞬时速度估计 自归一化重要性采样

在策略改进步骤中，平均流策略的目标分布是由软 Q 函数诱导的玻尔兹曼分布：

$$\pi(\mathbf{a}_0 | \mathbf{s}) = \frac{\exp(\frac{1}{\alpha} Q(\mathbf{s}, \mathbf{a}_0))}{Z(\mathbf{s})}, \quad (19)$$

其中 $Z(\mathbf{s})$ 表示归一化常数。在本小节中，我们描述如何优化平均速度网络以逼近该目标分布。

如公式 8 所示，训练平均流模型仅需要目标分布瞬时速度场的无偏估计量。该速度场可通过将条件速度场边缘化得到：

$$\mathbf{v}_t(\mathbf{a}_t | \mathbf{s}) = \mathbb{E}_{\pi(\mathbf{a}_0 | \mathbf{a}_t, \mathbf{s})} [\mathbf{v}_t(\mathbf{a}_t | \mathbf{a}_0, \mathbf{s})] \quad (20)$$

$$= \mathbb{E}_{\pi(\mathbf{a}_0, \mathbf{a}_1 | \mathbf{a}_t, \mathbf{s})} [\mathbf{a}_1 - \mathbf{a}_0]. \quad (21)$$

标准平均流训练从目标分布中采样，并计算相应的条件速度作为边缘速度场的无偏估计量。然而，在策略改进步骤中，目标分布由 Q 函数隐式定义，无法直接采样。为解决这一问题，我们进一步分析边缘速度场。将该关系代入上式可得

$$\mathbf{a}_1 = \frac{\mathbf{a}_t - (1-t)\mathbf{a}_0}{t}$$

$$\mathbf{v}_t(\mathbf{a}_t | \mathbf{s}) = \mathbb{E}_{\pi(\mathbf{a}_0 | \mathbf{a}_t, \mathbf{s})} \left[\frac{\mathbf{a}_t - \mathbf{a}_0}{t} \right]. \quad (22)$$

最大熵动态规划与软演员评论家算法 (Dong 等, 2025; Ma 等, 2025) 表明，与玻尔兹曼策略相关的条件分布具有如下形式：

$$\pi(\mathbf{a}_0 | \mathbf{a}_t, \mathbf{s}) \propto \exp(\frac{1}{\alpha} Q(\mathbf{s}, \mathbf{a}_0)) \mathcal{N}\left(\mathbf{a}_0 \left| \frac{\mathbf{a}_t}{1-t}, \left(\frac{t}{1-t}\right)^2 I\right.\right). \quad (23)$$

完整推导见附录 B.2。令 $q(\mathbf{a}_0) = \mathcal{N}(\mathbf{a}_0 | \frac{\mathbf{a}_t}{1-t}, (\frac{t}{1-t})^2 I)$ ，与最大熵动态规划类似，我们可以抽取样本 $\mathbf{a}_0 \sim q(\mathbf{a}_0)$ ，并使用自归一化重要性采样估计边缘速度场：

$$\mathbf{v}_t(\mathbf{a}_t | \mathbf{s}) = \mathbb{E}_{q(\mathbf{a}_0)} \left[\frac{\pi(\mathbf{a}_0 | \mathbf{a}_t, \mathbf{s})}{q(\mathbf{a}_0)} \cdot \frac{\mathbf{a}_t - \mathbf{a}_0}{t} \right] \quad (24)$$

$$\approx \sum_{i=1}^K \frac{w(\mathbf{a}_0^i)}{\sum_{j=1}^K w(\mathbf{a}_0^j)} \cdot \frac{\mathbf{a}_t - \mathbf{a}_0^i}{t}, \quad (25)$$

其中重要性权重 $w(\mathbf{a}_0) = \frac{\pi(\mathbf{a}_0 | \mathbf{a}_t, \mathbf{s})}{q(\mathbf{a}_0)} \propto \exp(\frac{1}{\alpha} Q(\mathbf{s}, \mathbf{a}_0))$ ， K 表示样本数量。

然而，该自归一化重要性采样估计量并非对所有 $t \in [0, 1]$ 都高效。其相对于使用来自 $\pi(\mathbf{a}_0|\mathbf{a}_t, \mathbf{s})$ 样本的蒙特卡洛估计量的相对效率可通过有效样本量量化，有效样本量衡量为保证蒙特卡洛估计量的方差与使用 K 个样本的自归一化重要性采样估计量相同所需的蒙特卡洛样本数 (Kong, 1992; Metelli 等, 2020)。有效样本量的简要介绍见附录 B.3。因此，在 K 个样本的预算下，有效样本量较大的自归一化重要性采样估计量具有更小的估计方差，效率更高。自归一化重要性采样的有效样本量可估计为

$$\widehat{\text{ESS}}(\pi||q) = \frac{\left(\sum_{i=1}^K w(\mathbf{a}_0^i)\right)^2}{\sum_{i=1}^K w^2(\mathbf{a}_0^i)}. \quad (26)$$

当提议分布 q 逐渐偏离目标分布 π 时，有效样本量从 K 退化至 1。因此，需要一个与目标分布足够接近的提议分布。根据式 23，当 t 接近零时，由于高斯提议的方差较小，目标分布高度集中，高斯提议与目标分布紧密匹配。相比之下，当 t 接近 1 时，高斯提议变得平坦得多，目标分布越来越受指数 Q 函数项支配，导致 q 与目标分布匹配不佳，从而有效样本量较低。

为了提高大 t 下的估计效率，本文引入第二个提议分布。该提议应满足：(i) 易于采样，(ii) 可进行似然计算，(iii) 接近由 Q 函数诱导的玻尔兹曼分布，而该分布被高斯提议所忽略。利用第 4.2 节开发的似然估计方法，当前策略分布 $\pi_\theta(\mathbf{a}_0|\mathbf{s})$ 满足上述三个要求。随着策略优化的进行，该分布越来越集中于高 Q 值区域，使其成为合适的提议。此外，为了根据效率自适应选择估计器，本文以有效样本量作为权重，将两个自归一化重要性采样估计器相结合。具体地，定义 $q^1(\mathbf{a}_0) = \pi_\theta(\mathbf{a}_0|\mathbf{s})$ 和

$q^2(\mathbf{a}_0) = \mathcal{N}\left(\mathbf{a}_0|\frac{\mathbf{a}_t}{1-t}, (\frac{t}{1-t})^2 I\right)$ 边缘速度场的最终估计器为

$$\hat{\mathbf{v}}_t(\mathbf{a}_t|\mathbf{s}) = \sum_{k=1}^2 \frac{\text{ESS}_k}{\sum_{l=1}^2 \text{ESS}_l} \hat{\mathbf{v}}_t^k(\mathbf{a}_t|\mathbf{s}), \quad (27)$$

其中每个 $\hat{\mathbf{v}}_t^k$ 使用式 25 计算，重要性权重为 $w^k(\mathbf{a}_0) = \frac{\pi(\mathbf{a}_0|\mathbf{a}_t, \mathbf{s})}{q^k(\mathbf{a}_0)}$ 。 $\pi(\mathbf{a}_0|\mathbf{a}_t, \mathbf{s})$ 的归一化常数在自归一化重要性采样和有效样本量计算中相互抵消，因此无需评估。所提估计器的详细偏差和方差分析见附录 B.4。

最后，平均流策略的训练目标为

$$\mathcal{L}(\theta) = \mathbb{E}_{\mathbf{s}, r, t, \mathbf{a}_t} \|\mathbf{u}_\theta(\mathbf{s}, \mathbf{a}_t, r, t) - \text{sg}(\mathbf{u}_{\text{tgt}})\|_2^2, \quad (28)$$

其中 $\mathbf{u}_{\text{tgt}} = \hat{\mathbf{v}}_t - (t-r)(\hat{\mathbf{v}}_t \partial_{\mathbf{a}} \mathbf{u}_\theta + \partial_t \mathbf{u}_\theta)$ 。

在图 1 中，本文实证评估了不同 t 值下两个提议分布的有效样本量和估计方差。结果表明，当 t 变大时，高斯提议 q^2 的有效样本量大幅下降，而策略提议 q^1 仍保持相对较高的有效样本量。通过结合两个自归一化重要性采样估计器，估计方差小于任一单独估计器。

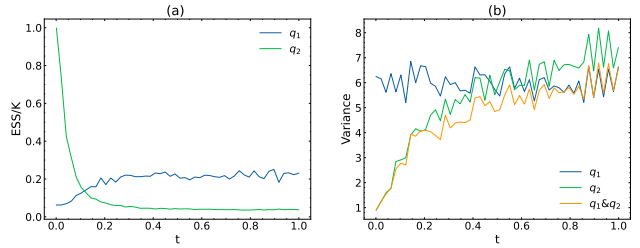


图 1. 两种自归一化重要性采样估计器在不同提议分布下的有效样本量与估计方差，在 HalfCheetah-v3 基准上经过 12 万次训练迭代后评估。(a) 归一化有效样本量（有效样本量除以样本数），数值越大表示提议分布越高效。(b) 估计器的估计方差，使用相同数量的样本计算。

4.4. 实用实现

在本小节中，我们提出所提出的平均流策略优化 (MFPO) 算法的实用实现。我们采用两个参数化神经网络： Q_ϕ 用于近似 Q 函数， $\mathbf{u}_\theta$ 用于表示平均流策略。该算法遵循软策略迭代框架，在策略评估和策略改进之间交替进行，以迭代优化平均流策略。

在策略评估步骤中，通过最小化软贝尔曼误差来训练 Q 函数

$$\mathcal{L}(\phi) = \mathbb{E}_{(\mathbf{s}, \mathbf{a}, \mathbf{s}') \sim \mathcal{B}} \left[\frac{1}{2} (Q_\phi(\mathbf{s}, \mathbf{a}) - Q_{\text{target}}(\mathbf{s}, \mathbf{a}))^2 \right], \quad (29)$$

其中 $\mathcal{B}$ 表示存储过去交互的回放缓冲区，目标 Q 值定义为 $Q_{\text{target}}(\mathbf{s}, \mathbf{a}) = r(\mathbf{s}, \mathbf{a}) + \gamma (Q_\phi(\mathbf{s}', \mathbf{a}') - \alpha \log \pi_\theta(\mathbf{a}'|\mathbf{s}'))$ 。在计算目标 Q 值时，下一时间步的动作及其对应似然根据算法 2 获得。

在策略改进步骤中，我们使用式 (28) 中定义的损失来优化平均流策略，该损失鼓励策略逼近由学习到的 Q 函数诱导的玻尔兹曼分布。尽管最小化器

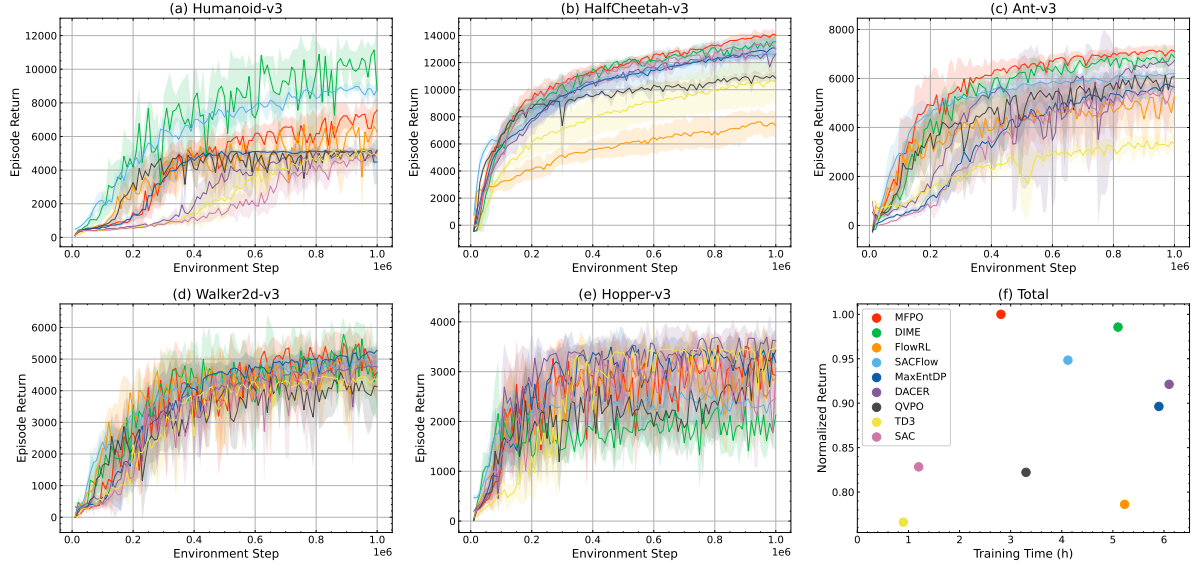


图 2. 不同算法在 5 个 MuJoCo 运动基准上的比较。(a – e) 在 5 个随机种子上的平均学习曲线；阴影区域表示标准差。(f) 所有任务的平均归一化回报与训练时间的关系。每个任务上的归一化回报通过将最后 10% 环境步的平均回报除以 MFPO 的平均回报来计算。越靠近左上区域的方法表现出更高的性能和更高的时间效率。

表 1. 不同算法的采样步数和每个样本的平均推理延迟，在 MuJoCo 基准上评估。

算法	MFPO	DIME	FlowRL	SAC-Flow	MaxEntDP	DACER	QVPO	TD3	SAC
Sampling Steps	2	16	1 ¹	4	20	20	20	1	1
Inference Time (ms)	0.46	0.97	0.42	0.96	1.56	1.06	1.68	0.14	0.15

该目标的最优解通常与式 (4) 中 KL 散度的最优解不一致，当玻尔兹曼分布位于策略类之外时 Π 。本文假设由于平均流策略类的表达能力，实际中两者的解是接近的。在更新策略网络之后，平均发散网络使用式 (17) 中的目标进行训练。完整的 MFPO 算法总结于算法 3。

此外，为了进一步提升 MFPO 的性能和稳定性，本文引入了以下技术。

自动调节温度。 温度系数 α 是一个关键的超参数，控制探索与利用之间的权衡。遵循 SAC (Haarnoja 等人, 2018)，本文自动调节 α 以匹配预定义的目标策略熵 $\mathcal{H}_{\text{target}}$ ，从而避免手动调节的需要。具体而言， α 通过最小化以下目标进行优化

$$\mathcal{L}(\alpha) = \alpha(\mathcal{H}(\pi_\theta) - \mathcal{H}_{\text{target}}), \quad (30)$$

其中策略熵 $\mathcal{H}(\pi_\theta)$ 通过对 MeanFlow 策略生成动作的负对数似然取平均来估计。

分布评论家。 通过显式建模分布

回报分布方面，分布式 Q 学习 (Bellemare 等, 2017; Duan 等, 2021) 已被证明能产生更稳定、更准确的策略评估。它在先前的基于扩散的方法中也展现了有效性 (Wang 等, 2024; Celik 等, 2025)。遵循 C51 (Bellemare 等, 2017)，我们将 Q 函数表示为离散回报值集合上的分类分布，并使用该分布的均值作为策略优化的最终标量 Q 值。

动作选择。 虽然随机策略在训练期间有利于探索，但在测试时可能不如确定性策略。为解决这一问题，我们采用动作选择技术 (Mao 等, 2024; Ding 等, 2024; Espinosa-Dice 等, 2025)。在评估期间，MeanFlow 策略生成多个候选动作，并选择具有最高估计 Q 值的动作与环境交互。

5. 实验

在本节中，我们评估所提出的 MFPO 方法的性能。我们在 5 个运动任务上进行了实验

¹FlowRL 采用中点 ODE 求解器；因此，一个采样步骤需要两次前向网络评估。

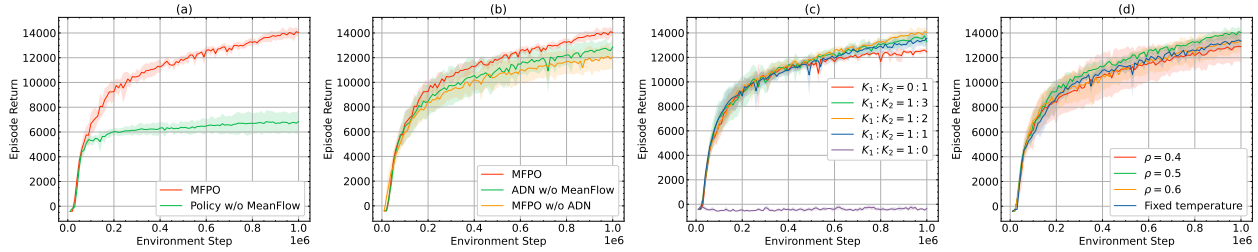


图 3. HalfCheetah-v3 基准上的消融研究。(a) 比较建模平均速度与瞬时速度的策略的学习曲线。(b) 使用平均散度网络 (ADN)、瞬时散度网络和无散度网络的动作似然估计的学习曲线。(c) 自适应瞬时速度估计中两种提议在不同采样比例下的学习曲线。(d) 不同目标熵系数 ρ 和固定温度下的学习曲线。固定温度设置为使用 $\rho = 0.5$ 获得的训练过程中的平均温度。

来自 MuJoCo (Todorov 等, 2012) 的任务、来自 DeepMind Control Suite (Tassa 等, 2018) 的 6 个困难任务, 以及来自 HumanoidBench (Sferrazza 等, 2024) 的 3 个高维任务。与最先进的基线算法的比较见第 5.1 节, 分析 MFPO 关键组件的消融研究报告于第 5.2 节。

5.1. 对比评估

为验证本文方法的优越性, 我们将 MFPO 与 6 种基于扩散的算法——DIME (Celik 等, 2025)、FlowRL (Lv 等, 2025)、SAC-Flow (Zhang 等, 2026)、MaxEntDP (Dong 等, 2025)、DACER (Wang 等, 2024) 和 QVPO (Ding 等, 2024) ——以及 2 种采用高斯策略和确定性策略的经典基线方法 SAC (Haarnoja 等, 2018) 和 TD3 (Fujimoto 等, 2018) 进行了比较。性能和训练时间如图 2、图 7 和图 8 所示, 采样步数和推理时间见表 1。结果表明, 在大多数任务上, MFPO 达到了与竞争方法相当或更优的性能。值得注意的是, 与基于扩散的策略相比, MFPO 将训练时间缩短了约 50%, 显著缩小了扩散策略与传统单步策略之间的效率差距。

5.2. 消融研究

平均流模型。MFPO 采用平均流模型进行策略表示和动作似然估计中的散度积分。为评估建模平均速度场的益处, 本文将上述平均流模型替换为标准流匹配模型。如图 3(a) 和 (b) 所示, 在策略或散度积分中替换平均流模型均导致性能显著下降。这些结果凸显了在少步设置中学习平均速度的重要性, 其能有效缓解离散化误差。

动作似然估计。策略熵评估

评估与策略提议的重要性权重计算依赖于精确的动作似然估计。为考察平均散度网络 (ADN) 的作用, 我们通过将动作似然表达式中的发散积分设为零来移除其贡献。相应结果如图 3(b) 所示。不含 ADN 的 MFPO 变体性能低于完整模型, 表明 ADN 所实现的精确动作似然估计对于稳定且有效的策略优化至关重要。

自适应瞬时速度估计。本文研究了自适应瞬时速度估计中两种提议分布之间样本比例的影响, 结果如图 3(c) 所示。将来自策略提议和高斯提议的 SNIS 估计量相结合, 可以得到更稳定的瞬时速度估计, 从而相比仅使用高斯提议获得更好的性能。相比之下, 仅依赖策略提议无法产生可行的策略, 因为该提议只有在当前策略已经能够生成高 Q 值动作时才有效, 而没有高斯提议的辅助则难以实现。此外, 本文观察到为高斯提议分配略多的样本 ($K_1 : K_2 = 1 : 2$) 会带来更好的性能。这可能是因为增加的样本预算使策略提议能够更快地变得有效。

自动调节温度。本文将目标策略熵设置为与动作空间维度成正比, 即 $\mathcal{H}_{\text{target}} = -\rho \cdot \dim(\mathcal{A})$, 其中 ρ 是一个与任务无关的常数, 用于控制探索与利用之间的权衡。不同 ρ 值以及固定温度下的性能如图 3(d) 所示。过高的 ρ 可能会抑制对潜在高回报动作区域的探索, 而过低的 ρ 可能会损害奖励最大化。经验上, 本文发现 $\rho = 0.5$ 在大多数任务中提供了一个稳健的选择。此外, 与使用固定温度相比, 固定温度可能在训练过程中随着奖励尺度增加而面临探索不足的问题, 而自动温度调节在整个训练过程中保持一致的探索水平, 有助于稳定的策略改进。

更多消融研究和超参数分析见附录 E.7 和 E.8。

6. 结论

本文提出了 MFPO，一种在最大熵强化学习框架内使用平均流模型作为策略表示的方法。通过采用少步平均流策略，MFPO 在快速推理的同时实现了高表达性，显著提高了基于扩散的强化学习算法的训练和推理速度。为了解决平均流策略的优化挑战，本文引入了一个平均散度网络来近似动作似然，并提出了一种自适应瞬时速度估计方法来构建策略损失。大量实验表明，MFPO 在训练时间、推理时间和采样步数显著减少的情况下，能够达到或超过基于扩散的基线方法。

局限性与未来工作。尽管 MFPO 所需的采样步数少于现有基于扩散的算法，但达到高性能仍需两步采样。未来工作将探索更先进的策略优化技术，或集成更强大的生成模型，以将采样步数减少至一步。

致谢

本研究受国家重点研发计划项目（编号 2021ZD0201504）及中国人工智能学会 - 青博盛基金（编号 2025CAAI-QBOSAN-06）资助。

影响声明

基于扩散的策略现已广泛应用于机器人控制的视觉-语言-动作（VLA）系统中，但常面临计算成本高昂的问题。通过优化少步均值流策略，本文方法在保持高性能的同时提升了训练与推理效率。这种效率提升使得机器人控制响应更快、更灵敏，并降低了计算与能耗需求。因此，该方法有望促进实际部署，并扩大先进机器人系统的可及性。

参考文献

Bellemare, M. G., Dabney, W., and Munos, R. A distributional perspective on reinforcement learning. In *International Conference on Machine Learning*, pp. 449–458. PMLR, 2017.

Brooks, T., Peebles, B., Holmes, C., DePue, W., Guo, Y., Jing, L., Schnurr, D., Taylor, J., Luhman, T., Luhman, E., et al. Video generation models as world simulators. *OpenAI Blog*, 1(8):1, 2024.

Celik, O., Li, Z., Blessing, D., Li, G., Palenicek, D., Peters, J., Chalvatzaki, G., and Neumann, G. Dime: Diffusion-based maximum entropy reinforcement learning. In *International Conference on Machine Learning*, 2025.

Chao, C.-H., Feng, C., Sun, W.-F., Lee, C.-K., See, S., and Lee, C.-Y. Maximum entropy reinforcement learning via energy-based normalizing flow. *Advances in Neural Information Processing Systems*, 37:56136–56165, 2024.

Chen, H., Lu, C., Ying, C., Su, H., and Zhu, J. Offline reinforcement learning via high-fidelity generative behavior modeling. In *International Conference on Learning Representations*, 2023.

Chen, R. T., Rubanova, Y., Bettencourt, J., and Duvenaud, D. K. Neural ordinary differential equations. *Advances in Neural Information Processing Systems*, 31, 2018.

Chen, T., Ma, H., Li, N., Wang, K., and Dai, B. One-step flow policy mirror descent. *arXiv preprint arXiv:2507.23675*, 2025.

Chen, Y., Li, H., and Zhao, D. Boosting continuous control with consistency policy. In *Proceedings of the 23rd International Conference on Autonomous Agents and Multiagent Systems*, pp. 335–344, 2024.

Chi, C., Xu, Z., Feng, S., Cousineau, E., Du, Y., Burchfiel, B., Tedrake, R., and Song, S. Diffusion policy: Visuomotor policy learning via action diffusion. *The International Journal of Robotics Research*, 44(10-11): 1684–1704, 2025.

Dhariwal, P. and Nichol, A. Diffusion models beat gans on image synthesis. *Advances in Neural Information Processing Systems*, 34:8780–8794, 2021.

Ding, S., Hu, K., Zhang, Z., Ren, K., Zhang, W., Yu, J., Wang, J., and Shi, Y. Diffusion-based reinforcement learning via q-weighted variational policy optimization. *Advances in Neural Information Processing Systems*, 37: 53945–53968, 2024.

Ding, S., Hu, K., Zhong, S., Luo, H., Zhang, W., Wang, J., Wang, J., and Shi, Y. Genpo: Generative diffusion models meet on-policy reinforcement learning. *Advances in Neural Information Processing Systems*, 2025.

Ding, Z. and Jin, C. Consistency models as a rich and efficient policy class for reinforcement learning. In *International Conference on Learning Representations*, 2024.

Dong, X., Cheng, J., and Zhang, X. S. Maximum entropy reinforcement learning with diffusion policy. In *International Conference on Machine Learning*, 2025.

- Duan, J., Guan, Y., Li, S. E., Ren, Y., Sun, Q., and Cheng, B. Distributional soft actor-critic: Off-policy reinforcement learning for addressing value estimation errors. *IEEE Transactions on Neural Networks and Learning Systems*, 33(11):6584–6598, 2021.
- Espinosa-Dice, N., Zhang, Y., Chen, Y., Guo, B., Oertell, O., Swamy, G., Brantley, K., and Sun, W. Scaling offline rl via efficient and expressive shortcut models. *Advances in Neural Information Processing Systems*, 2025.
- Fang, L., Liu, R., Zhang, J., Wang, W., and Jing, B. Diffusion actor-critic: Formulating constrained policy iteration as diffusion noise regression for offline reinforcement learning. In *International Conference on Learning Representations*, 2025.
- Frans, K., Hafner, D., Levine, S., and Abbeel, P. One step diffusion via shortcut models. In *International Conference on Learning Representations*, 2025.
- Frostig, R., Johnson, M. J., and Leary, C. Compiling machine learning programs via high-level tracing. In *SysML conference 2018*, 2019.
- Fujimoto, S., Hoof, H., and Meger, D. Addressing function approximation error in actor-critic methods. In *International Conference on Machine Learning*, pp. 1587–1596. PMLR, 2018.
- Geng, Z., Deng, M., Bai, X., Kolter, J. Z., and He, K. Mean flows for one-step generative modeling. *arXiv preprint arXiv:2505.13447*, 2025.
- Haarnoja, T., Tang, H., Abbeel, P., and Levine, S. Reinforcement learning with deep energy-based policies. In *International conference on machine learning*, pp. 1352–1361. PMLR, 2017.
- Haarnoja, T., Zhou, A., Abbeel, P., and Levine, S. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In *International Conference on Machine Learning*, pp. 1861–1870. Pmlr, 2018.
- Hansen-Estruch, P., Kostrikov, I., Janner, M., Kuba, J. G., and Levine, S. Idql: Implicit q-learning as an actor-critic method with diffusion policies. *arXiv preprint arXiv:2304.10573*, 2023.
- Ho, J., Jain, A., and Abbeel, P. Denoising diffusion probabilistic models. *Advances in Neural Information Processing Systems*, 33:6840–6851, 2020.
- Hutchinson, M. F. A stochastic estimator of the trace of the influence matrix for laplacian smoothing splines. *Communications in Statistics-Simulation and Computation*, 18(3):1059–1076, 1989.
- Kang, B., Ma, X., Du, C., Pang, T., and Yan, S. Efficient diffusion policies for offline reinforcement learning. *Advances in Neural Information Processing Systems*, 36: 67195–67212, 2023.
- Kong, A. A note on importance sampling using standardized weights. *University of Chicago, Dept. of Statistics, Tech. Rep.*, 348:14, 1992.
- Li, Z., Tang, S., and Azizan, N. Reverse flow matching: A unified framework for online reinforcement learning with diffusion and flow policies. *arXiv preprint arXiv:2601.08136*, 2026.
- Lillicrap, T. P., Hunt, J. J., Pritzel, A., Heess, N., Erez, T., Tassa, Y., Silver, D., and Wierstra, D. Continuous control with deep reinforcement learning. *arXiv preprint arXiv:1509.02971*, 2015.
- Lipman, Y., Chen, R. T., Ben-Hamu, H., Nickel, M., and Le, M. Flow matching for generative modeling. In *International Conference on Learning Representations*, 2023.
- Liu, X., Gong, C., and Liu, Q. Flow straight and fast: Learning to generate and transfer data with rectified flow. *arXiv preprint arXiv:2209.03003*, 2022.
- Lu, C., Zhou, Y., Bao, F., Chen, J., Li, C., and Zhu, J. Dpm-solver: A fast ode solver for diffusion probabilistic model sampling in around 10 steps. *Advances in Neural Information Processing Systems*, 35:5775–5787, 2022.
- Lu, C., Chen, H., Chen, J., Su, H., Li, C., and Zhu, J. Contrastive energy prediction for exact energy-guided diffusion sampling in offline reinforcement learning. In *International Conference on Machine Learning*, pp. 22825–22855. PMLR, 2023.
- Lu, C., Zhou, Y., Bao, F., Chen, J., Li, C., and Zhu, J. Dpm-solver++: Fast solver for guided sampling of diffusion probabilistic models. *Machine Intelligence Research*, pp. 1–22, 2025.
- Lv, L., Li, Y., Luo, Y., Sun, F., Kong, T., Xu, J., and Ma, X. Flow-based policy for online reinforcement learning. *Advances in Neural Information Processing Systems*, 2025.
- Ma, H., Chen, T., Wang, K., Li, N., and Dai, B. Efficient online reinforcement learning for diffusion policy. In *International Conference on Machine Learning*, 2025.
- Mao, L., Xu, H., Zhan, X., Zhang, W., and Zhang, A. Diffusion-dice: In-sample diffusion guidance for offline reinforcement learning. *Advances in Neural Information Processing Systems*, 37:98806–98834, 2024.

- Metelli, A. M., Papini, M., Montali, N., and Restelli, M. Importance sampling techniques for policy optimization. *Journal of Machine Learning Research*, 21(141):1–75, 2020.
- Owen, A. B. *Monte Carlo theory, methods and examples*. <https://artowen.su.domains/mc/>, 2013.
- Park, S., Li, Q., and Levine, S. Flow q-learning. In *International Conference on Machine Learning*, 2025.
- Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., et al. Pytorch: An imperative style, high-performance deep learning library. *Advances in neural information processing systems*, 32, 2019.
- Pearce, T., Rashid, T., Kanervisto, A., Bignell, D., Sun, M., Georgescu, R., Macua, S. V., Tan, S. Z., Momennejad, I., Hofmann, K., et al. Imitating human behaviour with diffusion models. In *International Conference on Learning Representations*, 2023.
- Psenka, M., Escontrela, A., Abbeel, P., and Ma, Y. Learning a diffusion model policy from rewards via q-score matching. In *International Conference on Machine Learning*, pp. 41163–41182. PMLR, 2024.
- Ren, A. Z., Lidard, J., Ankile, L. L., Simeonov, A., Agrawal, P., Majumdar, A., Burchfiel, B., Dai, H., and Simchowitz, M. Diffusion policy optimization. In *International Conference on Learning Representations*, 2025.
- Saharia, C., Chan, W., Saxena, S., Li, L., Whang, J., Denton, E. L., Ghasemipour, K., Gontijo Lopes, R., Karagol Ayan, B., Salimans, T., et al. Photorealistic text-to-image diffusion models with deep language understanding. *Advances in Neural Information Processing Systems*, 35: 36479–36494, 2022.
- Salimans, T. and Ho, J. Progressive distillation for fast sampling of diffusion models. In *International Conference on Learning Representations*, 2022.
- Sauer, A., Lorenz, D., Blattmann, A., and Rombach, R. Adversarial diffusion distillation. In *European Conference on Computer Vision*, pp. 87–103. Springer, 2024.
- Schulman, J., Levine, S., Abbeel, P., Jordan, M., and Moritz, P. Trust region policy optimization. In *International Conference on Machine Learning*, pp. 1889–1897. PMLR, 2015.
- Schulman, J., Wolski, F., Dhariwal, P., Radford, A., and Klimov, O. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- Sferrazza, C., Huang, D. M., Lin, X., Lee, Y., and Abbeel, P. Humanoidbench: Simulated humanoid benchmark for whole-body locomotion and manipulation. *arXiv preprint arXiv:2403.10506*, 2024.
- Skilling, J. The eigenvalues of mega-dimensional matrices. In *Maximum Entropy and Bayesian Methods: Cambridge, England, 1988*, pp. 455–466. Springer, 1989.
- Sohl-Dickstein, J., Weiss, E., Maheswaranathan, N., and Ganguli, S. Deep unsupervised learning using nonequilibrium thermodynamics. In *International Conference on Machine Learning*, pp. 2256–2265. pmlr, 2015.
- Song, Y. and Dhariwal, P. Improved techniques for training consistency models. In *International Conference on Learning Representations*, 2024.
- Song, Y. and Ermon, S. Generative modeling by estimating gradients of the data distribution. *Advances in Neural Information Processing Systems*, 32, 2019.
- Song, Y., Sohl-Dickstein, J., Kingma, D. P., Kumar, A., Ermon, S., and Poole, B. Score-based generative modeling through stochastic differential equations. In *International Conference on Learning Representations*, 2021.
- Song, Y., Dhariwal, P., Chen, M., and Sutskever, I. Consistency models. In *International Conference on Machine Learning*, pp. 32211–32252. PMLR, 2023.
- Tassa, Y., Doron, Y., Muldal, A., Erez, T., Li, Y., Casas, D. d. L., Budden, D., Abdolmaleki, A., Merel, J., Lefrancq, A., et al. Deepmind control suite. *arXiv preprint arXiv:1801.00690*, 2018.
- Todorov, E., Erez, T., and Tassa, Y. Mujoco: A physics engine for model-based control. In *2012 IEEE/RSJ International Conference on Intelligent Robots and Systems*, pp. 5026–5033. IEEE, 2012.
- Wang, Y., Wang, L., Jiang, Y., Zou, W., Liu, T., Song, X., Wang, W., Xiao, L., Wu, J., Duan, J., et al. Diffusion actor-critic with entropy regulator. *Advances in Neural Information Processing Systems*, 37:54183–54204, 2024.
- Wang, Z., Hunt, J. J., and Zhou, M. Diffusion policies as an expressive policy class for offline reinforcement learning. In *International Conference on Learning Representations*, 2023.
- Yang, L., Huang, Z., Lei, F., Zhong, Y., Yang, Y., Fang, C., Wen, S., Zhou, B., and Lin, Z. Policy representation via diffusion probability model for reinforcement learning. *arXiv preprint arXiv:2305.13122*, 2023.
- Zhang, S., Zhang, W., and Gu, Q. Energy-weighted flow matching for offline reinforcement learning. In *International Conference on Learning Representations*, 2025a.

Zhang, T., Yu, C., Su, S., and Wang, Y. Reinflow: Fine-tuning flow matching policy with online reinforcement learning. *Advances in Neural Information Processing Systems*, 2025b.

Zhang, Y., Yu, S., Zhang, T., Guang, M., Hui, H., Long, K., Wang, Y., Yu, C., and Ding, W. SAC flow: Sample-efficient reinforcement learning of flow-based policies via velocity-reparameterized sequential modeling. In *International Conference on Learning Representations*, 2026.

A. 补充相关工作

A.1. 面向离线与离线到在线强化学习的扩散策略。

在线强化学习范畴之外，扩散策略在离线及离线到在线强化学习场景中也展现出显著效能。本文在此对这些方法进行总结，以提供该领域更广阔的视角。在离线强化学习中，早期研究应用扩散策略来近似复杂的行为策略，这些策略可能表现出高度的多模态性和偏斜性（Pearce 等，2023；Chi 等，2025）。后续工作尝试提升扩散策略超越行为策略的性能。SfBC 和 IDQL（Chen 等，2023；Hansen-Estruch 等，2023）生成多个候选动作，并通过在其 Q 值上的 Softmax 分布中采样来选择动作。EDP 和 QIPO（Kang 等，2023；Zhang 等，2025a）利用 Q 函数的变换对扩散损失进行重新加权，以强调具有更高 Q 值的动作。DQL、CPQL、CAC 和 FQL（Wang 等，2023；Chen 等，2024；Ding 与 Jin，2024；Park 等，2025）引入了一个额外的 Q 损失，该损失在扩散策略生成的动作上进行评估，并与扩散损失联合优化。CEP 和 DAC（Lu 等，2023；Fang 等，2025）将 Q 引导项纳入扩散策略的得分函数中，引导采样过程朝向具有更高 Q 值的动作区域。遵循离线到在线强化学习范式，DPPO 和 ReFlow（Ren 等，2025；Zhang 等，2025b）进一步引入在线交互，以微调在离线数据集上预训练的扩散策略。他们将逆向扩散过程形式化为具有简单高斯策略的马尔可夫决策过程（MDP），并应用 PPO（Schulman 等，2017）算法进行求解。

A.2. 对 DIME 在少采样步数下优化目标的分析

DIME 算法（Celik 等，2025）推导出扩散策略熵的变分下界。随后，将最大熵目标中的策略熵替换为该下界，从而获得原始优化目标的下界。具体而言，熵下界由下式给出

$$\mathcal{H}(\pi(\mathbf{a}_0|\mathbf{s})) \geq \mathbb{E}_{\overleftarrow{\pi}} \left[\log \frac{\overrightarrow{\pi}(\mathbf{a}_{1:T}|\mathbf{s}, \mathbf{a}_0)}{\overleftarrow{\pi}(\mathbf{a}_{0:T}|\mathbf{s})} \right], \quad (31)$$

其中 $\overrightarrow{\pi}$ 和 $\overleftarrow{\pi}$ 分别表示前向扩散过程和逆向生成过程， T 为采样步数。考虑真实熵与变分下界之间的差距：

$$\Delta = \mathcal{H}(\pi(\mathbf{a}_0|\mathbf{s})) - \mathbb{E}_{\overleftarrow{\pi}} \left[\log \frac{\overrightarrow{\pi}(\mathbf{a}_{1:T}|\mathbf{s}, \mathbf{a}_0)}{\overleftarrow{\pi}(\mathbf{a}_{0:T}|\mathbf{s})} \right] \quad (32)$$

$$= \mathbb{E}_{\overleftarrow{\pi}} [D_{\text{KL}}(\overleftarrow{\pi}(\mathbf{a}_{1:T}|\mathbf{s}, \mathbf{a}_0) \parallel \overrightarrow{\pi}(\mathbf{a}_{1:T}|\mathbf{s}, \mathbf{a}_0))]. \quad (33)$$

在两个过程均满足马尔可夫假设的条件下，联合 KL 散度可分解为

$$\Delta = \sum_{t=1}^T \mathbb{E}_{\overleftarrow{\pi}} [D_{\text{KL}}(\overleftarrow{\pi}(\mathbf{a}_{t-1}|\mathbf{a}_t, \mathbf{s}) \parallel \overrightarrow{\pi}(\mathbf{a}_{t-1}|\mathbf{a}_t, \mathbf{s}))], \quad (34)$$

这是参数化逆向转移与其真实对应之间逐步 KL 散度的总和。根据扩散理论，真实的逆向转移仅在无穷小极限 $T \rightarrow \infty$ （下收敛于高斯分布（Sohl-Dickstein 等，2015；Ho 等，2020）。对于有限的 T ，真实的逆向转移通常偏离高斯分布，无法被高斯近似精确捕捉。由于逆向转移通常被参数化为高斯分布，每个 KL 项保持严格正值，导致在少步机制下存在不可忽略的差距和宽松的界。

我们在不同采样步数下对 DIME 进行了实证评估（图 4）。将步数从 16 减少到 2 会导致性能显著下降。相比之下，我们的 MFPO 方法在 $T = 2$ 下实现了高性能，这表明 DIME 的退化并非由少步扩散策略的有限表达能力引起，而是由变分目标的宽松性所致。这促使我们开发更适合少步扩散策略学习的替代算法。

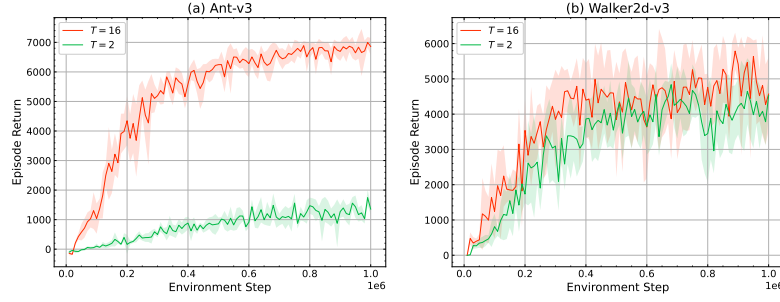


图 4. 使用不同采样步数的 DIME 算法学习曲线。

B. 理论推导

B.1. 平均散度网络的损失推导

真实平均散度定义为

$$\delta(\mathbf{s}, \mathbf{a}_t, r, t) = \frac{1}{t-r} \int_r^t \nabla \cdot \mathbf{v}_\theta(\mathbf{s}, \mathbf{a}_\tau, \tau) d\tau. \quad (35)$$

两边乘以 $t-r$ 并对 t 求导, 得到恒等式

$$\delta(\mathbf{s}, \mathbf{a}_t, r, t) = \nabla \cdot \mathbf{v}_\theta(\mathbf{s}, \mathbf{a}_t, t) - (t-r) \frac{d}{dt} \delta(\mathbf{s}, \mathbf{a}_t, r, t), \quad (36)$$

其中全时间导数由下式给出

$$\frac{d}{dt} \delta(\mathbf{s}, \mathbf{a}_t, r, t) = \mathbf{v}_\theta(\mathbf{s}, \mathbf{a}_t, t) \partial_{\mathbf{a}} \delta + \partial_t \delta. \quad (37)$$

本文取式 (36) 右端作为平均发散网络 δ_ω 的训练目标, 从而得到目标函数

$$\mathcal{L}(\omega) = \mathbb{E}_{\mathbf{s}, r, t, \mathbf{a}_t} \|\delta_\omega(\mathbf{s}, \mathbf{a}_t, r, t) - \text{sg}(\delta_{\text{tgt}})\|_2^2, \quad (38)$$

with

$$\delta_{\text{tgt}} = \nabla \cdot \mathbf{v}_\theta(\mathbf{s}, \mathbf{a}_t, t) - (t-r) (\mathbf{v}_\theta(\mathbf{s}, \mathbf{a}_t, t) \partial_{\mathbf{a}} \delta + \partial_t \delta). \quad (39)$$

实际中, 发散项由其无偏估计量替代, 瞬时速度由平均速度网络 $\mathbf{u}_\theta(\mathbf{s}, \mathbf{a}_t, r, t)$ 通过设定 $r=t$ 获得。因此最终训练目标变为

$$\delta_{\text{tgt}} = \widehat{\text{div}} - (t-r) (\mathbf{u}_\theta(\mathbf{s}, \mathbf{a}_t, t, t) \partial_{\mathbf{a}} \delta + \partial_t \delta), \quad (40)$$

其中 $\widehat{\text{div}}$ 表示式 (16) 中引入的斯基林-哈钦森迹估计器。

B.2. Rewriting the Conditional Distribution $\pi(\mathbf{a}_0 | \mathbf{a}_t, \mathbf{s})$

本节推导条件分布 $\pi(\mathbf{a}_0 | \mathbf{a}_t, \mathbf{s})$ 的等价表达式。具体地, 本文证明其可写为

$$\pi(\mathbf{a}_0 | \mathbf{a}_t, \mathbf{s}) \propto \exp\left(\frac{1}{\alpha} Q(\mathbf{s}, \mathbf{a}_0)\right) \mathcal{N}\left(\mathbf{a}_0 \left| \frac{\mathbf{a}_t}{1-t}, \left(\frac{t}{1-t}\right)^2 \mathbf{I} \right.\right). \quad (41)$$

证明。由贝叶斯法则, 可得

$$\pi(\mathbf{a}_0 | \mathbf{a}_t, \mathbf{s}) = \frac{\pi(\mathbf{a}_0 | \mathbf{s}) \pi(\mathbf{a}_t | \mathbf{a}_0)}{\pi(\mathbf{a}_t | \mathbf{s})} \quad (42)$$

$$\propto \pi(\mathbf{a}_0 | \mathbf{s}) \pi(\mathbf{a}_t | \mathbf{a}_0), \quad (43)$$

其中 $\pi(\mathbf{a}_t|\mathbf{s})$ 是关于 $\mathbf{a}_0$ 的常数。

在最大熵强化学习框架下，目标策略满足 $\pi(\mathbf{a}_0|\mathbf{s}) = \frac{1}{Z(\mathbf{s})} \exp\left(\frac{1}{\alpha} Q(\mathbf{s}, \mathbf{a}_0)\right)$ ，分函数。将该表达式代入式 (43) 可得

$$\pi(\mathbf{a}_0|\mathbf{a}_t, \mathbf{s}) \propto \exp\left(\frac{1}{\alpha} Q(\mathbf{s}, \mathbf{a}_0)\right) \pi(\mathbf{a}_t|\mathbf{a}_0). \quad (44)$$

回顾前向插值过程定义为 $\mathbf{a}_t = (1-t)\mathbf{a}_0 + t\mathbf{a}_1$, $\mathbf{a}_1 \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$, , 这意味着 $\pi(\mathbf{a}_t|\mathbf{a}_0) = \mathcal{N}(\mathbf{a}_t | (1-t)\mathbf{a}_0, t^2\mathbf{I})$. Note that $\mathcal{N}(\mathbf{a}_t | (1-t)\mathbf{a}_0, t^2\mathbf{I}) \propto \exp\left(-\frac{\|\mathbf{a}_t - (1-t)\mathbf{a}_0\|^2}{2t^2}\right)$, and $\mathcal{N}\left(\mathbf{a}_0 \left| \frac{\mathbf{a}_t}{1-t}, \left(\frac{t}{1-t}\right)^2 \mathbf{I}\right.\right) \propto \exp\left(-\frac{\|\mathbf{a}_0 - \frac{\mathbf{a}_t}{1-t}\|^2}{2\left(\frac{t}{1-t}\right)^2}\right)$, we observe

$$\mathcal{N}(\mathbf{a}_t | (1-t)\mathbf{a}_0, t^2\mathbf{I}) \propto \mathcal{N}\left(\mathbf{a}_0 \left| \frac{\mathbf{a}_t}{1-t}, \left(\frac{t}{1-t}\right)^2 \mathbf{I}\right.\right), \quad (45)$$

其中比例关系在常数与 $\mathbf{a}_0$ 无关的情况下成立。将该关系代回条件分布即可完成证明：

$$\pi(\mathbf{a}_0|\mathbf{a}_t, \mathbf{s}) \propto \exp\left(\frac{1}{\alpha} Q(\mathbf{s}, \mathbf{a}_0)\right) \mathcal{N}\left(\mathbf{a}_0 \left| \frac{\mathbf{a}_t}{1-t}, \left(\frac{t}{1-t}\right)^2 \mathbf{I}\right.\right). \quad (46)$$

□

B.3. 自归一化重要性采样有效样本量简介

给定目标分布 $p(\mathbf{x})$ 、提议分布 $q(\mathbf{x})$ 以及目标函数 $f(\mathbf{x})$ ，本文旨在估计其期望。为此，本文将普通蒙特卡洛 ($\mathbb{E}_p[f(\mathbf{x})]$) (Monte Carlo, MC) 估计器与自归一化重要性采样 (Self-Normalized Importance Sampling) 估计器进行对比。
(自归一化重要性采样) 估计器：

$$\hat{\mu}_{\text{MC}} = \frac{1}{K} \sum_{i=1}^K f(\mathbf{x}_i), \quad \mathbf{x}_i \sim p, \quad (47)$$

$$\hat{\mu}_{\text{SNIS}} = \sum_{i=1}^K \frac{w(\mathbf{x}_i)}{\sum_{j=1}^K w(\mathbf{x}_j)} f(\mathbf{x}_i), \quad \mathbf{x}_i \sim q, \quad (48)$$

其中 $w(\mathbf{x}) = p(\mathbf{x})/q(\mathbf{x})$ 表示重要性权重。自归一化重要性采样 (SNIS) 估计器的效率通常通过有效样本量 (Effective Sample Size, ESS) 来量化 (Kong, 1992; Metelli et al., 2020)，该指标刻画了从 p 中抽取的独立同分布样本数量，这些样本需要达到与从 q 中抽取的 K 个样本相同的估计方差。形式上，有效样本量通过方差比渐近定义：

$$\text{ESS}(p \parallel q) \approx K \frac{\text{Var}(\hat{\mu}_{\text{MC}})}{\text{Var}(\hat{\mu}_{\text{SNIS}})}. \quad (49)$$

在实际应用中，有效样本量通常使用基什估计量 (Kish's Estimator) 进行近似 (Kong, 1992)：

$$\widehat{\text{ESS}}(p \parallel q) = \frac{\left(\sum_{i=1}^K w(\mathbf{x}_i)\right)^2}{\sum_{i=1}^K w^2(\mathbf{x}_i)}. \quad (50)$$

该度量取值范围为 1 至 K 。当 K 时达到最大值 $q = p$ ，并随着提议分布与目标分布之间差异的增大而降低，表明估计方差增大且采样效率下降。

B.4. 自适应瞬时速度估计器的偏差与方差分析

地面真值瞬时速度由下式给出： $\mathbf{v}_t(\mathbf{a}_t|\mathbf{s}) = \mathbb{E}_{\pi(\mathbf{a}_0|\mathbf{a}_t, \mathbf{s})} [\frac{\mathbf{a}_t - \mathbf{a}_0}{t}]$ 。本文考虑两个自归一化重要性采样 (SNIS) 估计器， $\hat{\mathbf{v}}_t^1$ 和 $\hat{\mathbf{v}}_t^2$ ，分别使用建议分布 $q^1(\mathbf{a}_0)$ 和 $q^2(\mathbf{a}_0)$ 构造，样本数分别为 K_1 和 K_2 。为了利用不同建议分布的互补优势，本文将组合估计器定义为两个 SNIS 估计器的线性组合：

$$\hat{\mathbf{v}}_t(\alpha) = \alpha \hat{\mathbf{v}}_t^1 + (1 - \alpha) \hat{\mathbf{v}}_t^2, \quad \alpha \in [0, 1]. \quad (51)$$

渐近偏差。众所周知，SNIS 估计器在有限样本下是有偏的，但渐近无偏。具体而言，对于每个估计器 $k \in \{1, 2\}$ ，其期望满足 (Owen, 2013)：

$$\mathbb{E}[\hat{\mathbf{v}}_t^k] = \mathbf{v}_t + \mathcal{O}(K_k^{-1}). \quad (52)$$

因此，作为权重之和为 1 的线性组合，组合估计器 $\hat{\mathbf{v}}_t(\alpha)$ 保持渐近无偏，偏差以 $\mathcal{O}(\min(K_1, K_2)^{-1})$ 的速率消失。

方差。假设两个 SNIS 估计器相互独立，组合估计器的方差为：

$$\text{Var}[\hat{\mathbf{v}}_t(\alpha)] = \alpha^2 \text{Var}[\hat{\mathbf{v}}_t^1] + (1 - \alpha)^2 \text{Var}[\hat{\mathbf{v}}_t^2]. \quad (53)$$

对式 (53) 关于 α 求最小化，得到最优权重：

$$\alpha^* = \frac{\text{Var}[\hat{\mathbf{v}}_t^2]}{\text{Var}[\hat{\mathbf{v}}_t^1] + \text{Var}[\hat{\mathbf{v}}_t^2]}. \quad (54)$$

因此，最小方差估计器应赋予方差较小的分量更大的权重。对于 SNIS，方差近似与有效样本量 (ESS) 成反比：

$$\text{Var}[\hat{\mathbf{v}}_t^k] \propto \frac{1}{\text{ESS}_k}. \quad (55)$$

将此近似代入式 (54)，我们推导出 MFPO 中采用的自适应有效样本量加权估计器：

$$\hat{\mathbf{v}}_t = \frac{\text{ESS}_1}{\text{ESS}_1 + \text{ESS}_2} \hat{\mathbf{v}}_t^1 + \frac{\text{ESS}_2}{\text{ESS}_1 + \text{ESS}_2} \hat{\mathbf{v}}_t^2. \quad (56)$$

该有效样本量加权估计器是渐近无偏的，并且在两个自归一化重要性采样估计器的线性组合中达到近最优方差。因此，该估计器自动偏向于与目标分布更匹配的建议分布，为 MFPO 提供稳定且高效的瞬时速度估计。

B.5. MFPO 的收敛性分析

软策略评估。我们首先证明，在给定精确似然估计器的情况下，最小化软贝尔曼误差能够恢复当前策略的软 Q 函数。

命题 B.1 (软策略评估)。假设式 (17) 中的平均散度网络 (ADN) 损失被最小化，使得所得 ADN 恢复当前 MeanFlow 策略的真实动作似然 $\log \pi_\theta(\mathbf{a}|\mathbf{s})$ 。进一步假设式 (29) 中的软贝尔曼误差在足够表达力的函数类上被最小化。则学习到的 Q 函数 Q_ϕ 收敛到当前策略 π_θ 的真实软 Q 函数 Q^{π_θ} 。

证明。对于 MeanFlow 策略，动作的对数似然可通过瞬时变量替换公式计算，该公式依赖于传输轨迹上瞬时速度场的散度。Skilling – Hutchinson 迹估计器为该散度提供无偏估计。因此，当 ADN 损失

$$\mathcal{L}(\omega) = \mathbb{E}_{\mathbf{s}, r, t, \mathbf{a}_t, \epsilon_{1:N}} \|\delta_\omega(\mathbf{s}, \mathbf{a}_t, r, t) - \text{sg}(\delta_{\text{tgt}})\|_2^2$$

被最小化时，学习到的 ADN δ_ω 恢复相应的平均发散。因此，所得的似然估计 $\log \pi_\theta(\mathbf{a}|\mathbf{s})$ 是准确的。

给定当前策略的准确似然，软贝尔曼目标为

$$Q_{\text{target}}(\mathbf{s}, \mathbf{a}) = r(\mathbf{s}, \mathbf{a}) + \gamma \mathbb{E}_{\mathbf{s}' \sim p(\cdot | \mathbf{s}, \mathbf{a}), \mathbf{a}' \sim \pi_\theta(\cdot | \mathbf{s}')} [Q_\phi(\mathbf{s}', \mathbf{a}') - \alpha \log \pi_\theta(\mathbf{a}' | \mathbf{s}')].$$

因此，最小化软贝尔曼误差

$$\mathcal{L}(\phi) = \mathbb{E}_{(\mathbf{s}, \mathbf{a}, \mathbf{s}') \sim \mathcal{B}} \left[\frac{1}{2} (Q_\phi(\mathbf{s}, \mathbf{a}) - Q_{\text{target}}(\mathbf{s}, \mathbf{a}))^2 \right]$$

强制 Q_ϕ 成为与 π_θ 相关的软贝尔曼算子的不动点。由于该算子是一个压缩映射（Haarnoja 等人，2017），它存在唯一的不动点，即软 Q 函数 Q^{π_θ} 。因此，在全局最小值处， $Q_\phi = Q^{\pi_\theta}$ 。 $\square$

软策略改进。接下来我们证明，当自适应瞬时速度估计（AIVE）无偏时，策略改进步骤恢复由当前软 Q 函数诱导的玻尔兹曼策略。

命题 B.2（软策略改进）。设 K_1 和 K_2 分别表示 AIVE 中使用的两个 SNIS 估计器的样本量。假设 $K_1, K_2 \rightarrow \infty$ ，使得 AIVE 提供与目标分布相关的瞬时速度场的无偏估计器

$$\pi^*(\mathbf{a} | \mathbf{s}) = \frac{\exp\left(\frac{1}{\alpha} Q^{\pi_{\text{old}}}(\mathbf{s}, \mathbf{a})\right)}{Z(\mathbf{s})}.$$

进一步假设平均流策略类 Π 具有足够的表达能力，使得 $\pi^*(\cdot | \mathbf{s}) \in \Pi$ 。那么最小化式 (28) 中的平均流策略损失可得到 $\pi_{\text{new}} = \pi^*$ ，这在最大熵目标下优于 π_{old} 。

证明。根据附录 B.4 中 AIVE 的偏差分析，当 $K_1, K_2 \rightarrow \infty$ 时，AIVE 提供了对应于向目标策略 $\pi^*(\cdot | \mathbf{s})$ 传输的瞬时速度场的无偏估计。因此，最小化平均流策略损失 $\mathcal{L}(\theta) = \mathbb{E}_{\mathbf{s}, r, t, \mathbf{a}_t} \|\mathbf{u}_\theta(\mathbf{s}, \mathbf{a}_t, r, t) - \text{sg}(\mathbf{u}_{\text{tgt}})\|_2^2$ 可恢复平均速度场，其诱导的终端分布为 $\pi^*(\cdot | \mathbf{s})$ 。因此，更新后的平均流策略满足

$$\pi_{\text{new}}(\cdot | \mathbf{s}) = \pi^*(\cdot | \mathbf{s}).$$

根据标准的软策略改进定理，由 $Q^{\pi_{\text{old}}}$ 诱导的玻尔兹曼策略 π^* 最小化式 (4) 中的 KL 目标，因此满足 $Q^{\pi_{\text{new}}}(\mathbf{s}, \mathbf{a}) \geq Q^{\pi_{\text{old}}}(\mathbf{s}, \mathbf{a})$ ， $\forall (\mathbf{s}, \mathbf{a})$ ，

在最大熵目标下。因此，策略改进步骤优于 π_{old} 。 $\square$

软策略迭代。根据第 3.2 节中的标准软策略迭代定理，通过在软策略评估和软策略改进之间反复交替，MFPO 将在 Π 内收敛到最优最大熵策略。

C. 算法伪代码

我们分别在算法 1、2 和 3 中给出了平均散度网络的训练过程、似然感知采样过程以及完整的 MFPO 训练算法。

D. 实验细节

本文所有实验均在一台配备 NVIDIA GeForce RTX 3090 GPU 和 Intel Xeon Platinum 8280 CPU 的服务器上进行。基线方法的实现遵循其官方代码库：DIME 1、FlowRL 2，¹ <https://github.com/ALRhub/DIME> ²

<https://github.com/bytedance/FlowRL>

Algorithm 1 Training of the Average Divergence Network

- 1: **Input:** policy network u_θ ; average divergence network δ_ω ; a batch of B state-action pairs $\{(s, a_0)\}$ from the current policy
- 2: Sample $(r, t) \sim p(r, t)$ and $a_1 \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
- 3: $a_t \leftarrow (1 - t)a_0 + ta_1$
- 4: **for** each a_t in the batch **do**
- 5: Sample $\{\epsilon_i\}_{i=1}^N \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
- 6: Estimate divergence: $\widehat{\text{div}}(s, a_t, t) = \frac{1}{N} \sum_{i=1}^N \epsilon_i^\top \frac{\partial u_\theta(s, a_t, t)}{\partial a_t} \epsilon_i$
- 7: Construct target: $\delta_{\text{tgt}}(s, a_t, r, t) = \widehat{\text{div}} - (t - r)(u_\theta(s, a_t, t) \partial_a \delta + \partial_t \delta)$
- 8: **end for**
- 9: Update parameters: $\omega \leftarrow \arg \min_\omega \frac{1}{B} \sum \|\delta_\omega(s, a_t, r, t) - \text{sg}(\delta_{\text{tgt}}(s, a_t, r, t))\|_2^2$

Algorithm 2 MeanFlow Action Sampling and Likelihood Estimation

- 1: **Input:** state s ; policy network u_θ ; average divergence network δ_ω ; number of sampling steps T ; prior distribution p_1
- 2: Sample $a_{t_T} \sim p_1(a_1)$
- 3: Initialize $\log p(a_{t_T}) \leftarrow \log p_1(a_1)$
- 4: **for** $i = T, \dots, 1$ **do**
- 5: $a_{t_{i-1}} \leftarrow a_{t_i} - \frac{1}{T} u_\theta(s, a_{t_i}, t_{i-1}, t_i)$
- 6: $\log p(a_{t_{i-1}}) \leftarrow \log p(a_{t_i}) + \frac{1}{T} \delta_\omega(s, a_{t_i}, t_{i-1}, t_i)$
- 7: **end for**
- 8: **Output:** action a_0 and log-likelihood $\log p(a_0)$

SAC-Flow³、MaxEntDP⁴、DACER⁵、QVPO⁶、TD3⁷ 和 SAC⁸。这些官方实现使用不同的机器学习框架开发。具体而言，DIME、SAC-Flow、MaxEntDP 和 DACER 使用 JAX (Frostig 等人, 2019) 实现，FlowRL 使用 PyTorch (Paszke 等人, 2019) 实现并以编译模式运行，性能与 JAX 相当，而 QVPO、SAC 和 TD3 使用 PyTorch 实现。为确保训练和推理效率的公平比较，QVPO、SAC 和 TD3 的训练和推理时间均使用我们基于 JAX 的实现进行测量。所有算法共享的超参数汇总于表 2。所有超参数均遵循官方实现中的默认设置，但批大小、网络规模和更新数据比在所有方法中统一以进行公平比较。

表 2. 所有算法共享的超参数。

Hyperparameter	MFPO	DIME	FlowRL	SAC-Flow	MaxEntDP	DACER	QVPO	TD3	SAC
Batch size	256	256	256	256	256	256	256	256	256
Discount factor γ	0.99	0.99	0.99	0.99	0.99	0.99	0.99	0.99	0.99
Target smoothing coefficient τ	0.005	1	0.95	1	0.005	0.005	0.005	0.005	0.005
No. of hidden layers	3	3	3	N/A	3	3	3	3	3
No. of hidden nodes per layer	256	256	256	N/A	256	256	256	256	256
Actor learning rate	3e-4	3e-4	3e-4	3e-4	3e-4	1e-4	3e-4	3e-4	3e-4
Critic learning rate	3e-4	3e-4	3e-4	1e-3	3e-4	1e-4	3e-4	3e-4	3e-4
Activation function	gelu	gelu	elu	gelu	mish	mish	mish	relu	relu
Replay buffer size	1e6	1e6	1e6	1e6	1e6	1e6	1e6	1e6	1e6
Update-to-data ratio	1	1	1	1	1	1	1	1	1
Diffusion steps	2	16	1	4	20	20	20	N/A	N/A
No. of action candidates	10	N/A	N/A	N/A	10	N/A	32	N/A	N/A

³<https://github.com/Elessar123/SAC-FLOW>
⁴<https://github.com/diffusionyes/MaxEntDP>
⁵<https://github.com/happy-yan/DACER-Diffusion-with-Online-RL>
⁶<https://github.com/wadx2019/qvpo>
⁷<https://github.com/sfujim/TD3>
⁸<https://github.com/toshikwa/soft-actor-critic.pytorch>

Algorithm 3 Mean Flow Policy Optimization (MFPO)

```

1: Input: Q-network  $Q_\phi$ ; target Q-network  $Q_{\bar{\phi}}$ ; policy network  $u_\theta$ ; average divergence network  $\delta_\omega$ ; temperature  $\alpha$ 
2: for each iteration do
3:   for each interaction step do
4:     Sample  $\mathbf{a} \sim \pi_\theta(\cdot|\mathbf{s})$  using Algorithm 2
5:     Execute action and observe  $(\mathbf{s}', r) = \text{env}(\mathbf{a})$ 
6:     Store transition  $(\mathbf{s}, \mathbf{a}, r, \mathbf{s}')$  in replay buffer  $\mathcal{B}$ 
7:   end for
8:   for each update step do
9:     Sample  $B$  transitions  $\{(\mathbf{s}, \mathbf{a}, r, \mathbf{s}')\}$  from  $\mathcal{B}$ 
10:    // Policy evaluation:
11:    Sample  $\mathbf{a}' \sim \pi_\theta(\cdot|\mathbf{s}')$  and compute  $\log \pi_\theta(\mathbf{a}'|\mathbf{s}')$  using Algorithm 2
12:    Compute target:  $Q_{\text{target}} \leftarrow r + \gamma (Q_{\bar{\phi}}(\mathbf{s}', \mathbf{a}') - \alpha \log \pi_\theta(\mathbf{a}'|\mathbf{s}'))$ 
13:    Update Q-network:  $\phi \leftarrow \arg \min_\phi \frac{1}{B} \sum \frac{1}{2} (Q_\phi(\mathbf{s}, \mathbf{a}) - Q_{\text{target}})^2$ 
14:    // Policy improvement:
15:    Sample  $(r, t) \sim p(r, t)$ 
16:    for  $k = 1, 2$  do
17:      Sample  $\mathbf{a}_0 \sim q^k$ 
18:      Estimate instantaneous velocity via SNIS:  $\hat{\mathbf{v}}_t^k = \sum \frac{w(\mathbf{a}_0)}{\sum w(\mathbf{a}_0)} \cdot \frac{\mathbf{a}_t - \mathbf{a}_0}{t}$ 
19:      Compute effective sample size:  $\text{ESS}_k = \frac{(\sum w(\mathbf{a}_0))^2}{\sum w^2(\mathbf{a}_0)}$ 
20:    end for
21:    Combine estimates:  $\hat{\mathbf{v}}_t = \frac{\text{ESS}_1}{\text{ESS}_1 + \text{ESS}_2} \hat{\mathbf{v}}_t^1 + \frac{\text{ESS}_2}{\text{ESS}_1 + \text{ESS}_2} \hat{\mathbf{v}}_t^2$ 
22:    Construct target:  $\mathbf{u}_{\text{tgt}} = \hat{\mathbf{v}}_t - (t - r) (\hat{\mathbf{v}}_t \partial_{\mathbf{a}} \mathbf{u}_\theta + \partial_t \mathbf{u}_\theta)$ 
23:    Update policy network:  $\theta \leftarrow \arg \min_\theta \frac{1}{B} \sum \|\mathbf{u}_\theta(\mathbf{s}, \mathbf{a}_t, r, t) - \text{sg}(\mathbf{u}_{\text{tgt}})\|_2^2$ 
24:    Update the average divergence network  $\delta_\omega$  using Algorithm 1
25:    Update target network:  $\bar{\phi} \leftarrow \tau \phi + (1 - \tau) \bar{\phi}$ 
26:  end for
27: end for
    
```

E. 补充实验

E.1. 二维玩具示例

我们在一个二维玩具示例上进行实验，以验证所提出的 MFPO 方法的有效性。具体而言，策略网络和平均散度网络被训练以近似一个具有 6 个对称分布分量的高斯混合模型（Gaussian Mixture Model, GMM），其中奖励函数定义为目标 GMM 的对数概率。训练后，我们将 MeanFlow 策略生成的动作及其由平均散度网络获得的对数似然估计可视化，如图 5(c) 所示。作为参考，图 5(a) 展示了从真实 GMM 采样的动作及其精确对数似然，而图 5(b) 展示了通过使用 1000 步欧拉求解器求解扩散常微分方程获得的动作和对数似然估计。可以观察到，2 步 MFPO 产生的动作分布和对数似然估计与真实 GMM 和细粒度欧拉求解器的结果高度吻合。这一结果表明，MFPO 仅需少量采样步骤即可准确近似目标动作分布及其似然，凸显了其有效性。

E.2. MFPO 关键组件的计算时间

我们评估了 MFPO 中关键组件的计算开销和性能增益，包括平均流模型、平均散度网络（ADN）和自适应瞬时速度估计（AIVE）。我们从一个扩散策略基线开始，该基线通过移除所有三个组件获得，其中算法学习一个标准 Q 函数，并训练一个瞬时速度网络来在单一高斯提议下近似 SNIS 估计器。然后我们逐步加入每个组件，以评估其在计算成本和策略性能方面的贡献。所有变量的回报均以完整 MFPO 算法的回报进行归一化。结果报告在表 3 中。总体而言，所提出的组件仅带来很小的计算开销，同时提供了显著的性能提升。

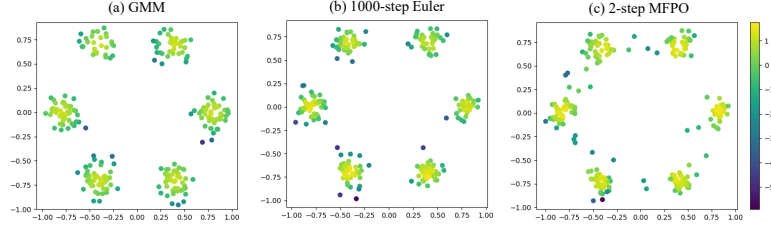


图 5. 二维玩具示例上生成动作及其对数似然估计的可视化。每个点表示一个动作，颜色表示相应的对数似然值。(a) 来自真实 GMM 的动作和精确对数似然。(b) 通过使用 1000 步欧拉求解器求解扩散 ODE 获得的动作和对数似然估计。(c) 由 2 步 MFPO 产生的动作和对数似然估计。

improvements.

表 3. MFPO 中关键组件的训练开销和性能增益，在 Mujoco 基准上评估。

Algorithm	Training Time (h)	Normalized Return
Diffusion Policy	2.45	0.52
MeanFlow Policy	2.45	0.77
MeanFlow Policy + ADN	2.58	0.91
MeanFlow Policy + ADN + AIVE	2.81	1.00

E.3. 平均散度网络的近似误差

有效的软策略评估关键依赖于平均散度网络（ADN）对动作似然的准确估计。在图 6 中，我们报告了 ADN 的训练损失、Skilling – Hutchinson 迹估计器的方差，以及 ADN 产生的动作似然估计相对于 1000 步欧拉 ODE 求解器所得结果的相对误差，后者作为近真实参考。结果表明，训练损失和估计器方差在整个训练过程中均保持较低水平，表明优化稳定。此外，ADN 仅使用两个采样步骤就能紧密匹配 1000 步欧拉求解器的似然估计，从而为软策略评估实现了高效且准确的似然估计。

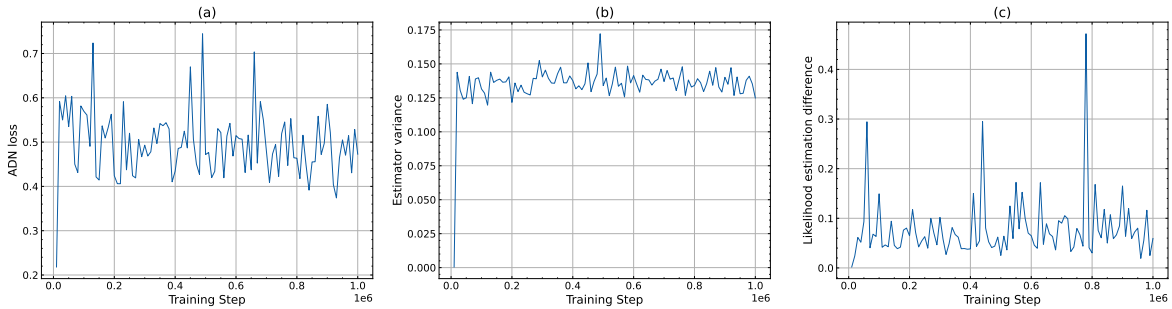


图 6. 平均散度网络（ADN）在 HalfCheetah-v3 基准上的训练分析。(a) ADN 的训练损失。(b) Skilling – Hutchinson 迹估计器的估计方差。(c) 动作似然估计相对于 1000 步欧拉 ODE 求解器的相对误差，以后者进行归一化。

E.4. DeepMind Control Suite 与 HumanoidBench 上的对比评估

本文将 MFPO 与基线算法在 DeepMind Control Suite 的 6 个困难任务和 HumanoidBench 的 3 个高维控制任务上进行比较。如图 7 和图 8 所示，MFPO 始终达到与基于扩散的基线相当或更优的性能，同时所需训练时间更少。

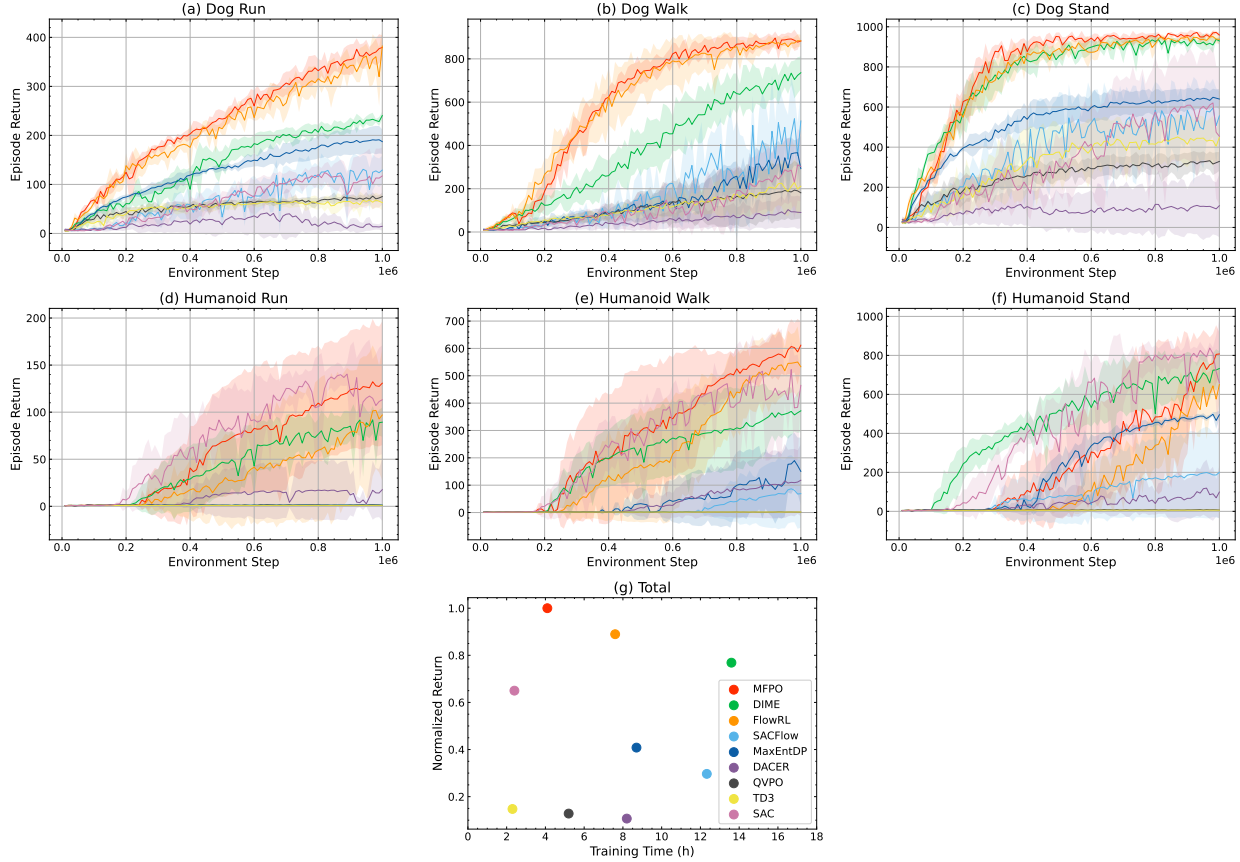


图 7. 不同算法在 DeepMind Control Suite 6 个困难任务上的比较。(a – f) 学习曲线为 5 个随机种子上的平均值；阴影区域表示标准差。(g) 所有任务的平均归一化回报与训练时间的关系。每个任务的归一化回报通过将最后 10% 环境步的平均回报除以 MFPO 的对值得到。越靠近左上角的方法表示性能越高且时间效率越高。

E.5. 与高斯混合模型和归一化流的比较

除了本文采用的 MeanFlow 模型外，其他生成模型（如高斯混合模型和归一化流）也能表示多模态分布。本文将 MFPO 与基于 GMM 的策略和基于归一化流的策略进行比较，以展示 MeanFlow 模型作为策略表示的优势。具体而言，本文实现了 SAC-GMM，即 SAC 的一种变体，其策略参数化为高斯混合模型，并分别使用 4 个和 16 个混合分量进行评估。本文还与 MEow (Chao 等人, 2024) 进行比较，后者是一种使用归一化流表示策略的最大熵强化学习算法。

如图 9 所示，MFPO 优于 SAC-GMM 和 MEow。对于基于 GMM 的策略，合适的混合分量数量依赖于具体任务，且在实践中难以确定。对于归一化流，可逆变换的要求限制了网络参数化的类别，这可能限制了其表达能力。这些局限性可能部分解释了观察到的性能差距，并凸显了 MeanFlow 模型作为最大熵强化学习中表达性策略表示的有效性。

E.6. AntMaze 基准上的多模态策略学习

在本小节中，我们研究 MFPO 能否在复杂的多目标强化学习任务上学习多模态策略。为了突出将最大熵强化学习目标与表达性策略表示相结合的重要性，我们将 MFPO 与 SAC 和 BPTT 进行了比较。其中，BPTT 是我们自行实现的一个简单基线，它在标准强化学习目标下，通过直接沿流链反向传播 Q 梯度来训练 MeanFlow 策略。我们在 AntMaze 基准上评估这些算法，该基准要求算法控制四足机器人从起点导航到任意目标。

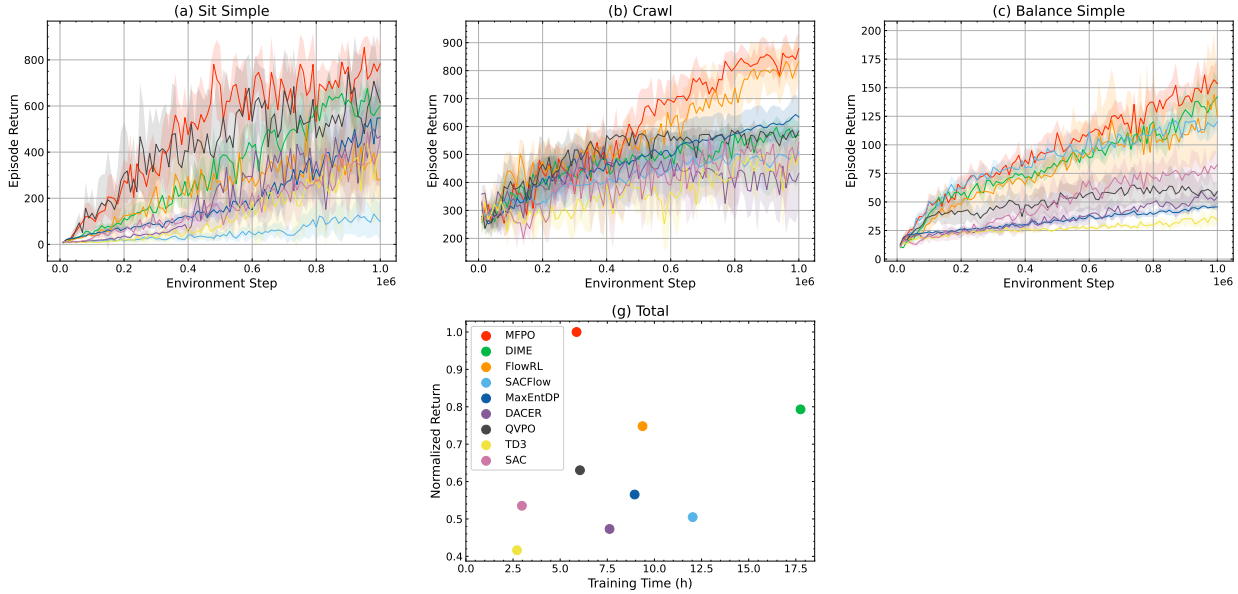


图 8. 不同算法在 HumanoidBench 中 3 个高维任务上的比较。(a-f) 学习曲线为 5 个随机种子平均结果；阴影区域表示标准差。(g) 所有任务的平均归一化回报与训练时间的关系。每个任务的归一化回报通过将最后 10% 环境步数的平均回报除以 MFPO 的对应值计算得到。越靠近左上角的方法表现出更高的性能和更高的时间效率。

图 10 展示了不同算法生成的轨迹。MFPO 发现了多条可行路径，证明了其在具有挑战性的多目标任务中学习多模态策略的能力。尽管 SAC 优化最大熵强化学习目标以鼓励多样化行为，但高斯策略的单峰性质将其限制在单一行为模式。因此，SAC 可能仅收敛到一个可行解，甚至无法找到有效解，正如在 AntMaze-v4 上观察到的那样。这些结果表明，能够建模复杂多模态动作分布的富有表现力的策略表示对于有效探索多种行为模式至关重要。

然而，仅凭表达能力是不够的。若缺乏熵正则化，由于纯粹的 Q 值最大化，BPTT 往往会坍缩至近似确定性的解。事实上，在标准强化学习目标下，最优策略会将所有概率质量集中在使 $Q(s, a)$ 最大化的动作上，而非维持多样化的动作模式。总体而言，MeanFlow 策略与最大熵强化学习框架的结合是必要的：MeanFlow 模型提供了表示多模态行为所需的表达性策略类，而最大熵目标则防止模式崩塌并鼓励发现多样化的可行解。

E.7. 消融研究

分布式 Q 学习。 由于策略改进步骤由策略评估期间获得的 Q 函数引导，准确估计当前策略的 Q 函数对于有效的策略优化至关重要。如图 11(a) 所示，禁用分布式 Q 学习会导致 MFPO 的性能下降。这一结果表明，通过分布式 Q 学习提高策略评估的准确性在 MFPO 的整体性能中发挥着重要作用。

动作选择。 在推理时，动作选择技术可以进一步提高 MeanFlow 策略生成动作的最优性。我们在图 11(b) 中评估了不同动作候选数量 M 的影响。结果表明，采用动作选择 ($M > 1$) 优于不采用动作选择 ($M = 1$) 的设置。因此，我们在所有实验中设置 $M = 10$ ，因为它能在仅产生适度计算开销的情况下实现强劲的性能。

E.8. 超参数分析

这里我们分析关键超参数对 MFPO 性能的影响。

采样步数。 通过对平均速度场建模，MeanFlow 策略即使在采样步数较少的情况下也能保持高表达能力。如图 12(a) 所示，仅使用两个采样步数就能达到与

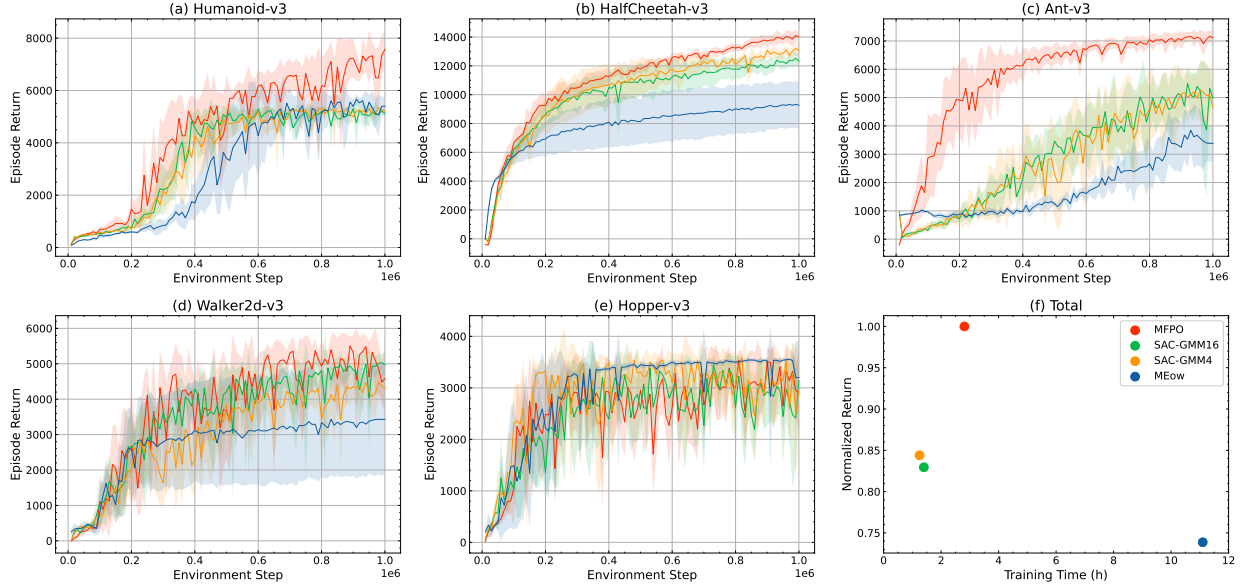


图 9. 在 MuJoCo 基准上与 SAC-GMM 和 MEow 的比较。(a – f) 学习曲线在 5 个随机种子上的平均值；阴影区域表示标准差。(g) 所有任务的平均归一化回报与训练时间的关系。每个任务的归一化回报通过将最后 10% 环境步数的平均回报除以 MFPO 的平均回报来计算。越靠近左上区域的方法表现出更高的性能和更高的时间效率。

使用更多步数（例如， $T = 16$ ）所获得的性能相当，这证明了采用 MeanFlow 模型作为策略表示的优势。因此，在本文中，我们将 $T = 2$ 设为默认配置。

瞬时速度估计的样本数量。增加样本数量可同时降低自归一化重要性采样（SNIS）估计器的偏差与方差，因此在计算资源允许的情况下更为可取。这一趋势与图 12(b) 中的实证结果一致，其中较大的样本量带来了性能的提升。基于这一权衡，本文设置 $K_1 = 16$ 和 $K_2 = 32$ ，它们在大多数任务中表现良好。

散度估计的样本数量。散度估计器的方差随着样本数量 N 的增加而减小。因此，使用更多样本可获得更稳定的散度估计，进而稳定平均散度网络的训练并改善动作似然估计。如图 12(c) 所示，设置 $N = 2$ 已能取得较强性能，而进一步增加 N 仅带来边际提升。

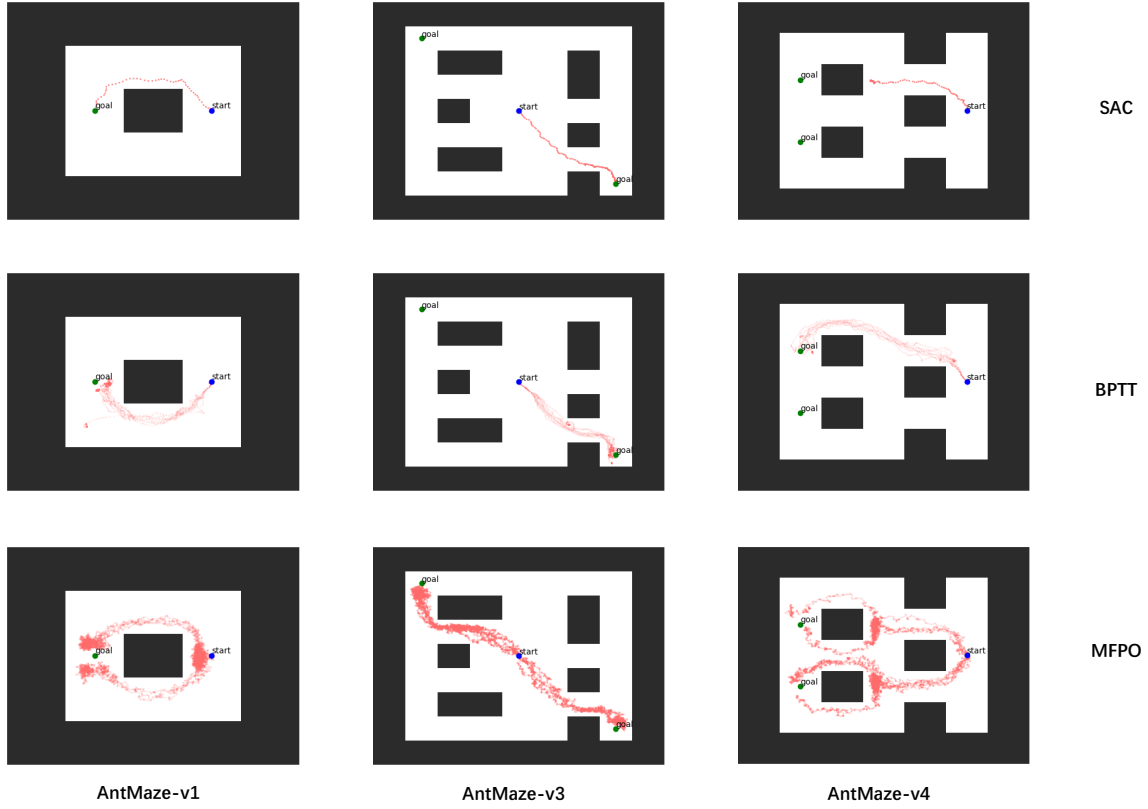


图 10. 在 AntMaze 任务上经过 100k 环境交互后, SAC、BPTT 和 MFPO 生成的轨迹。

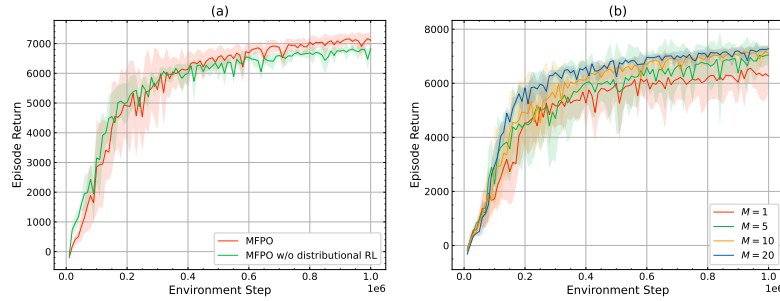


图 11. 在 Ant-v3 基准上对分布式 Q 学习和动作选择的消融实验。(a) 有和无分布式 Q 学习的 MFPO 学习曲线。(b) 不同候选动作数量下 MFPO 的学习曲线。

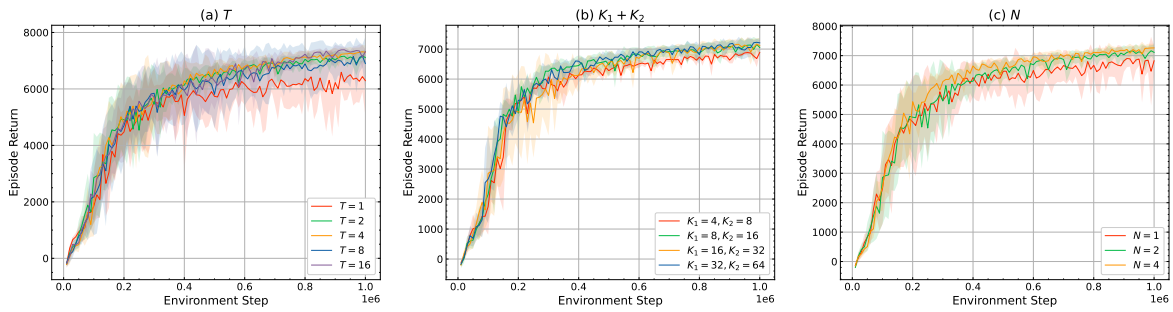


图 12. 在 Ant-v3 基准上的超参数分析。(a) 不同采样步数 T 下 MFPO 的学习曲线。(b) 不同瞬时速度估计样本数量下 MFPO 的学习曲线; 两个提议分布的样本比例固定, 仅改变其总数。(c) 不同散度估计样本数量 N 下 MFPO 的学习曲线。